

尼恩说在前面

在40岁老架构师 尼恩的**读者交流群**(50+)中，很多小伙伴拿到了一线互联网企业如得物、阿里、滴滴、极兔、有赞、希音、百度、网易、美团的面试机会，遇到很多很重要的面试题：

- 1.请解释ThreadLocal是什么，以及它的主要用途是什么？
- 2.ThreadLocal的内部机制是怎样的？请解释一下ThreadLocalMap和Entry。
- 3.使用ThreadLocal是否会导致内存泄漏？如果是，如何避免？
- 4.在使用线程池时，ThreadLocal可能会出现什么问题？如何解决？
- 5.能否解释一下 TransmittableThreadLocal 与ThreadLocal的区别和联系？
- 6.在父子线程间如何共享数据？ThreadLocal能实现吗？如果不能，那应如何实现？

最近有小伙伴在面试字节，又遇到了ThreadLocal 相关的面试题。小伙伴懵了，支支吾吾的说了几句，面试官不满意，面试挂了。

所以，尼恩给大家做一下系统化、体系化的梳理，使得大家内力猛增，可以充分展示一下大家雄厚的“技术肌肉”，**让面试官爱到“不能自己、口水直流”**，然后实现“offer直提”。

这里，尼恩团队把 ThreadLocal、ITL、TTL、FTL进行了全面的梳理，穿透式的梳理，

梳理为一个PDF文档《ThreadLocal 学习圣经：一次穿透TL、ITL、TTL、FTL》，并且持续迭代。

这个文档将成为大家 面试的杀手锏，此文当最新PDF版本，可以找40岁老架构师尼恩获取。



当然，上面的面试题以及参考答案，也会收入咱们的《[尼恩Java面试宝典PDF](#)》V171版本，供后面的小伙伴参考，提升大家的 3高 架构、设计、开发水平。

最新《尼恩 架构笔记》《尼恩高并发三部曲》《尼恩Java面试宝典》的PDF，请关注本公众号【技术自由圈】获取，回复：领电子书

本文作者：

- 第一作者 Moen （负责写初稿）
- 第二作者 尼恩（40岁老架构师，负责提升此文的技术高度，让大家有一种 俯视 技术的感觉）

本文目录

尼恩说在前面

本文目录

什么是ThreadLocal（TL）？

ThreadLocal的基本使用

ThreadLocal的成员方法

ThreadLocal的作用和优劣势

作用

优势

劣势

ThreadLocal的使用场景

场景1：使用ThreadLocal进行线程隔离

场景2：使用ThreadLocal进行跨函数数据传递

场景3：ThreadLocal在Java框架中的应用

ThreadLocal综合使用案例

ThreadLocal 使用总结

ThreadLocal的实现原理

ThreadLocal内部结构演进

ThreadLocalMap对象和Entry是什么

ThreadLocal的结构模型

ThreadLocal源码分析

set(T value)方法

get()方法

remove()方法

initialValue()方法

ThreadLocalMap源码分析

ThreadLocalMap的主要成员变量

Entry的Key需要使用弱引用

ThreadLocal造成内存泄露的问题

ThreadLocal是怎么造成内存泄露的呢？

ThreadLocal 有两个引用链

情况1：key的泄漏

情况2：value的泄漏

情况1的解决方案：使用弱引用，解决key的内存泄露

情况2的解决方案：清理策略解决value内存泄露

源码：value的 探测式清理：

源码：value的启发式清理：

业务主动清理：手动清除解决内存泄露

大总结：ThreadLocal的内存泄露与解决方案

ThreadLocal与内存泄露：防范与诊断

使用ThreadLocal的性能问题和优化措施

ThreadLocal的性能开销

编程规范：推荐使用 static final 修饰ThreadLocal对象

ThreadLocal对象实例存在强引用，会导致三个彻底失效：

ThreadLocal升级版1：InheritableThreadLocal 可继承本地变量

什么是可继承本地变量InheritableThreadLocal（ITL）？

InheritableThreadLocal的基本使用

InheritableThreadLocal的原理分析

InheritableThreadLocal所带来的问题

ThreadLocal升级版2: TransmittableThreadLocal 可透传本地变量
什么是TransmittableThreadLocal (TTL) ?
TTL 使用场景
TransmittableThreadLocal的原理分析
TransmittableThreadLocal的应用场景
ThreadLocal、InheritableThreadLocal与TransmittableThreadLocal的比较
ThreadLocal和synchronized之间的比较
FastThreadLocal (FTL) 的实现原理
什么是FastThreadLocal (FTL) ?
FastThreadLocal 如何使用
FastThreadLocal 的优势
FastThreadLocal 为什么快
FastThreadLocal 源码分析
get方法
set方法
FastThreadLocal 的回收机制
FastThreadLocal 在Netty中的应用
和ThreadLocal相比, FastThreadLocal 的优势:
说在最后: 有问题找老架构取经
部分历史案例

什么是ThreadLocal (TL) ?

在Java的多线程并发执行过程中, 为保证多个线程对变量的安全访问, 可以将变量放到ThreadLocal类型的对象中, 使变量在每个线程中都有独立值, 不会出现一个线程读取变量时而被另一个线程修改的现象。

ThreadLocal类通常被翻译为“[线程本地变量](#)”, 或者“[线程局部变量](#)”。

ThreadLocal的英文字面翻译为“[线程本地](#)”, 实质上ThreadLocal代表的是线程本地变量, 可能将其命名为 [ThreadLocalVariable](#) 会更加容易让人理解。

以下来至官网的解释:

This class provides thread-local variables. These variables differ from their normal counterparts in that each thread that accesses one (via its get or set method) has its own, independently initialized copy of the variable. ThreadLocal instances are typically private static fields in classes that wish to associate state with a thread (e.g., a user ID or Transaction ID).

简单翻译如下:

此类提供 [线程局部变量](#)。这些变量与其正常对应变量的不同之处在于, 每个访问一个变量 (通过其get或set方法) 的线程都有 [自己的](#)、[独立初始化的变量副本](#)。ThreadLocal实例通常是类中的 [私有静态字段](#), 这些字段希望将状态与线程 (例如, 用户ID或事务ID) 相关联。

总结重点如下:

1. ThreadLocal 提供了一种访问某个变量的特殊方式: 访问到的变量属于当前线程, 即保证每个线程的变量不一样, 而同一个线程在任何地方拿到的变量都是当前这个线程私有的, 这就是所谓的线程隔离。
2. 如果要使用 ThreadLocal, 通常定义为 `private static` 类型, 根据编程范式最好是定义为 `private static final` 类型。

ThreadLocal的基本使用

ThreadLocal是位于JDK的java.lang核心包中。

如果程序创建了一个ThreadLocal实例，那么在访问这个变量的值时，每个线程都会拥有一个独立的、自己的本地值。

“线程本地变量” 可以看成专属于线程的变量，不受其他线程干扰，保存着线程的专属数据。

当线程结束后，每个线程所拥有的那一个本地值也会被释放。

在多线程并发操作“线程本地变量”时候，线程各自操作的是自己的本地值，从而规避了线程安全问题。

ThreadLocal如何是做到为每个线程存有一份独立的本地值的呢？

一个ThreadLocal实例可以形象地理解为一个Map（早期版本的ThreadLocal是这样设计的）。当工作线程Thread实例向本地变量保持某个值时，会以“Key-Value对”的形式保存在ThreadLocal内部的Map中，其中**Key为线程Thread实例**，**Value为待保存的值**。当工作线程Thread实例从ThreadLocal本地变量取值时，会以Thread实例为Key，获取其绑定的Value。

一个ThreadLocal实例内部结构的形象展示，大致如图1-18所示。

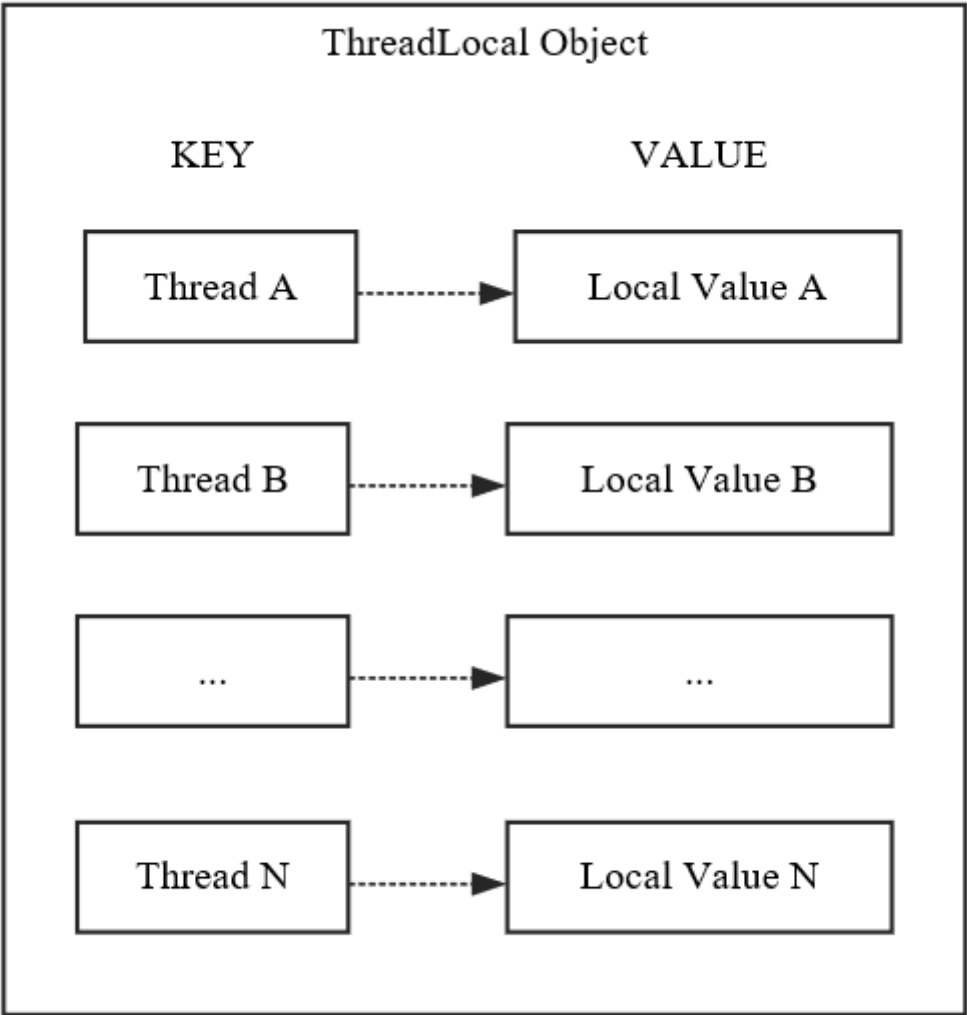


图1-18 一个ThreadLocal（早期版本）实例内部结构的形象展示

Java程序可以使用ThreadLocal的成员方法进行本地值操作，具体的成员方法如表1-2所示。

表1-2 ThreadLocal的成员方法

方法	说明
set(T value)	设置当前线程在“线程本地变量”实例中绑定的本地值
T get()	获得当前线程在“线程本地变量”实例中绑定的本地值
remove()	移除当前线程在“线程本地变量”实例中绑定的本地值

下面的例子，通过ThreadLocal的成员方法进行“线程本地变量”中线程本地值的设置、获取、移除，具体的代码如下：

```
package com.crazymakercircle.mutithread.basic.threadlocal;
...省略import
public class ThreadLocalTest
{
    @Data
    static class Foo
    {
        //实例总数
        static final AtomicInteger AMOUNT = new AtomicInteger(0);
        //对象的编号
        int index = 0;
        //对象的内容
        int bar = 10;
        //构造器
        public Foo()
        {
            index = AMOUNT.incrementAndGet(); //总数增加，并且给对象的编号
        }
        @Override
        public String toString()
        {
            return index + "@Foo{bar=" + bar + '}';
        }
    }

    //定义线程本地变量
    private static final ThreadLocal<Foo> LOCAL_FOO = new ThreadLocal<Foo>();

    public static void main(String[] args) throws InterruptedException
    {
        //获取自定义的混合型线程池
        ThreadPoolExecutor threadPool =
            ThreadUtil.getMixedTargetThreadPool();

        //提交5个任务，将会用到5个线程
        for (int i = 0; i < 5; i++)
        {
            threadPool.execute(new Runnable()
            {
                @Override
                public void run()
                {
                    //获取“线程本地变量”中当前线程所绑定的值
                    if (LOCAL_FOO.get() == null)
```

```

    {
        //设置“线程本地变量”中当前线程所绑定的值
        LOCAL_FOO.set(new Foo());
    }

    Print.tco("初始的本地值: " + LOCAL_FOO.get());
    //每个线程执行10次
    for (int i = 0; i < 10; i++)
    {
        Foo foo = LOCAL_FOO.get();
        foo.setBar(foo.getBar() + 1); //值增1
        sleepMilliseconds(10);
    }
    Print.tco("累加10次之后的本地值: " + LOCAL_FOO.get());

    //删除“线程本地变量”中当前线程所绑定的值
    LOCAL_FOO.remove(); //这点对于线程池中的线程尤其重要
}
});
}
}
}

```

运行以上示例，其结果如下：

```

[apppool-1-mixed-3]: 初始的本地值: 3@Foo{bar=10}
[apppool-1-mixed-4]: 初始的本地值: 4@Foo{bar=10}
[apppool-1-mixed-5]: 初始的本地值: 5@Foo{bar=10}
[apppool-1-mixed-2]: 初始的本地值: 1@Foo{bar=10}
[apppool-1-mixed-1]: 初始的本地值: 2@Foo{bar=10}
[apppool-1-mixed-1]: 累加10次之后的本地值: 2@Foo{bar=20}
[apppool-1-mixed-3]: 累加10次之后的本地值: 3@Foo{bar=20}
[apppool-1-mixed-5]: 累加10次之后的本地值: 5@Foo{bar=20}
[apppool-1-mixed-2]: 累加10次之后的本地值: 1@Foo{bar=20}
[apppool-1-mixed-4]: 累加10次之后的本地值: 4@Foo{bar=20}

```

通过输出的结果可以看出，在“线程本地变量”（LOCAL_FOO）中，每一个线程都绑定了一个独立的值（Foo对象），这些值对象是线程的私有财产，可以理解为线程的本地值。

每一次操作都是在自己的同一个本地值上进行，例子中通过线程本地值的index始终一致可以看出，每个线程操作的是同一个Foo对象。

如果线程尚未在本地变量（如LOCAL_FOO）中绑定了一个值，直接通过get()方法去获取本地值，会获取到一个空值，此时可以通过set()方法设置一个值作为初始值，具体的代码如下所示：

```

//获取“线程本地变量”中当前线程所绑定的值
if (LOCAL_FOO.get() == null)
{
    //设置“线程本地变量”中当前线程所绑定的初始值
    LOCAL_FOO.set(new Foo());
}

```

在当前线程尚未绑定值时，如果希望能从线程本地变量获取初始值，而且也不想采用以上的“判空后设值”的繁琐方式，则可以在定义使用 ThreadLocal.withInitial(...) 静态工厂方法，

使用ThreadLocal.withInitial(...)的方式，可以在定义ThreadLocal对象时设置一个获取初始值的回调函数，具体的代码如下所示：

```
ThreadLocal<Foo> LOCAL_FOO = ThreadLocal.withInitial(() -> new Foo());
```

以上代码并没有使用new ThreadLocal()去构造一个ThreadLocal对象，而是使用withInitial(...)工厂方法创建一个ThreadLocal对象，并传递了一个获取初始值的Lambda回调函数。

在线程尚未绑定值而直接从“线程本地变量”获取值时，将会取得回调函数被调用之后所返回的值。

ThreadLocal的成员方法

Java程序可以使用ThreadLocal的成员方法进行本地值操作，具体的成员方法如下所示：

- set(T value)：设置当前线程在“线程本地变量”实例中绑定的本地值
- T get()：获得当前线程在“线程本地变量”实例中绑定的本地值
- remove()：移除当前线程在“线程本地变量”实例中绑定的本地值
- initialValue()：当“线程本地变量”在当前线程的ThreadLocalMap中尚未绑定值时，该方法用于获取初始值。

ThreadLocal的作用和优劣势

ThreadLocal的作用和优劣势主要体现在以下几个方面：

作用

1. **线程本地存储**：ThreadLocal为每个线程提供各自的变量副本，每个线程都可以读取和修改自己线程的本地变量。这意味着在多线程环境中，不同线程对ThreadLocal变量的操作是独立的，不会互相干扰。
2. **简化线程间数据传递**：通过使用ThreadLocal，可以将某些需要在多线程之间共享但又需要避免竞态条件的数据封装起来，每个线程访问的都是自己的数据副本，从而简化了线程间数据传递的复杂性。
3. **管理线程特定资源**：ThreadLocal常用于存储线程上下文信息，如用户会话信息、事务信息等，这些信息通常与特定线程关联，不需要在多个线程之间共享。

优势

1. **线程安全**：由于每个线程操作的是自己的变量副本，因此避免了多线程访问共享变量时可能出现的竞态条件和数据不一致问题，从而保证了线程安全。
2. **简化编程模型**：通过使用ThreadLocal，开发者可以更加专注于业务逻辑的实现，而不需要过多关注线程间数据同步和共享的问题，降低了编程复杂度。
3. **性能优化**：由于避免了线程间数据同步的开销，以及减少了不必要的锁竞争，因此在某些场景下，使用ThreadLocal可以提高系统的并发性能。

劣势

1. **内存消耗**：ThreadLocal为每个线程创建变量副本，这意味着当线程数量较多时，会占用较多的内存资源。特别是在长时间运行的系统中，如果线程频繁创建和销毁，可能会导致内存泄漏问题。
2. **数据共享限制**：由于ThreadLocal变量是线程私有的，因此无法直接实现线程间的数据共享。如果需要在线程间传递数据，可能需要借助其他机制（如消息队列、共享内存等）。
3. **使用不当可能导致内存泄露问题**：如果开发者在使用ThreadLocal时不小心忘记在线程结束后清理变量（例如通过调用remove()方法），那么这些变量可能会一直存在于内存中，造成内存泄露。此外，如果多个线程需要访问和修改同一份数据，那么ThreadLocal可能并不适合，因为它提供的是每个线程私有的变量副本。
4. **性能开销**：在ThreadLocal中ThreadLocalMap是一种使用线性探测法实现的哈希表，底层采用数组存储数据。ThreadLocal.set()/get()方法在数据密集时很容易出现Hash冲突，hash冲突使用的是线性探测法，需要O(n)时间复杂度解决冲突问题，效率较低。

综上所述

ThreadLocal在简化线程间数据传递、管理线程特定资源和提高线程安全性方面具有优势，但也需要注意其可能带来的内存消耗和数据共享限制等问题。在使用ThreadLocal时，应根据具体的应用场景和需求进行权衡和选择。

ThreadLocal的使用场景

ThreadLocal是解决线程安全问题一个较好方案，它通过为每个线程提供一个独立的本地值，去解决并发访问的冲突问题。很多情况下，使用ThreadLocal比直接使用同步机制（如 synchronized）解决线程安全问题更简单，更方便，且结果程序拥有更高的并发性。

ThreadLocal使用场景，大致可以分为以下两类：

1. 线程隔离

- ThreadLocal的主要价值在于线程隔离，ThreadLocal中数据只属于当前线程，其本地值对别的线程是不可见的，在多线程环境下，可以防止自己的变量被其他线程篡改。另外，由于各个线程之间的数据相互隔离，避免同步加锁带来的性能损失，大大提升了并发性的性能。
- ThreadLocal在线程隔离的最常用案例为：可以每个线程绑定一个用户会话信息、数据库连接、HTTP请求等，这样一个线程的所有调用到的处理函数都可以非常方便地访问这些资源。
- 常见的ThreadLocal使用场景为数据库连接独享、Session数据管理等场景在“线程隔离”场景中使用ThreadLocal的典型案例为：可以每个线程绑定一个数据库连接，是的这个数据库连接为线程所独享，从而避免数据库连接被混用而导致操作异常问题。

2. 跨函数传递数据

- 通常用于同一个线程内，跨类、跨方法传递数据时，如果不用ThreadLocal，那么相互之间的数据传递势必要靠返回值和参数，这样无形之中增加了这些类或者方法之间的耦合度。

场景1：使用ThreadLocal进行线程隔离

ThreadLocal在“线程隔离”应用场景的典型应用为“数据库连接独享”。下面的代码来自Hibernate，代码中通过ThreadLocal进行数据库连接（Session）的“线程本地化”存储，主要的代码如下：

```
private static final ThreadLocal threadSession = new ThreadLocal();

public static Session getSession() throws InfrastructureException {
    Session s = (Session) threadSession.get();
    try {
        if (s == null) {
            s = sessionFactory().openSession();
            threadSession.set(s);
        }
    } catch (HibernateException ex) {
        throw new InfrastructureException(ex);
    }
    return s;
}
```

Hibernate 对数据库连接进行了封装，一个Session 代表一个数据库连接。通过以上代码可以看到，在Hibernate的getSession()方法中，首先判断当前线程中有没有放进去session，

如果还没有，那么通过sessionFactory().openSession()来创建一个Session，再将Session 设置到ThreadLocal变量中，这个Session 相当于线程的私有变量，而不是所有线程共用的，显然其他线程中是取不到这个Session。

一般来说，完成数据库操作之后程序会将 session 关闭，从而节省数据库连接资源。如果Session的使用方式为共享而不是独占，在这种情况下，Session是多线程共享使用的，如果某个线程使用完成之后，直接将Session关闭，其他线程在操作Session就会报错。所以Hibernate通过ThreadLocal非常简单实现了数据库连接的安全使用。

场景2：使用ThreadLocal进行跨函数数据传递

ThreadLocal在“跨函数数据传递”应用场景的典型有很多：

- 用来传递请求过程中的用户ID。
- 用来传递请求过程中的用户会话（Session）。
- 用来传递HTTP的用户请求实例HttpRequest。
- 其他需要在函数之间频繁传递的数据。

以下代码来自于疯狂创客圈社群的微服务脚手架 Crazy-SpringCloud 工程，通过ThreadLocal在函数之间传递用户信息、会话信息等，并且封装成了一个独立的SessionHolder类，具体的代码如下：

```
package com.crazymaker.springcloud.common.context;
...省略import
public class SessionHolder
{
    // session id 线程本地变量
    private static final ThreadLocal<String> sidLocal =
        new ThreadLocal<>("sidLocal");

    // 用户信息 线程本地变量
    private static final ThreadLocal<UserDTO> sessionUserLocal =
        new ThreadLocal<>("sessionUserLocal");

    // session 线程本地变量
    private static final ThreadLocal<HttpSession> sessionLocal =
        new ThreadLocal<>("sessionLocal");
    ...省略其他

    /**
     * 保存session在线程本地变量中
     */
    public static void setSession(HttpSession session)
    {
        sessionLocal.set(session);
    }

    /**
     * 取得绑定在线程本地变量中的session
     */
    public static HttpSession getSession()
    {
        HttpSession session = sessionLocal.get();
        Assert.notNull(session, "session 未设置");
        return session;
    }
    ...省略其他
}
```

场景3: ThreadLocal在Java框架中的应用

1. Spring

- 在Spring框架中, ThreadLocal用于存储数据库连接等线程特定的资源。由于数据库连接是线程不安全的, 因此每个线程都需要有自己的连接副本。Spring通过ThreadLocal将数据库连接与当前线程关联起来, 从而避免了多线程环境下的数据竞争和不一致问题。
- 在Spring的事务管理中, ThreadLocal也扮演着重要角色。它确保了每个线程都有自己的事务上下文, 包括事务状态、回滚点等信息, 从而实现了事务的隔离性。

2. MyBatis

- MyBatis是一个优秀的持久层框架, 它支持定制化SQL、存储过程以及高级映射。在MyBatis中, ThreadLocal可以用来存储SqlSession对象。由于SqlSession不是线程安全的, 因此每个线程都应该拥有自己独立的SqlSession实例。通过ThreadLocal, MyBatis可以方便地实现SqlSession的线程局部存储, 确保每个线程都能正确地执行SQL操作。

3. 分布式系统

- 在分布式系统中, ThreadLocal可以用来传递全局ID和分支ID等关键信息。这些ID对于分布式事务的追踪和诊断至关重要。通过将这些ID存储在ThreadLocal中, 可以确保它们在整个请求处理过程中都能被正确传递和使用。

4. 日志框架

- 一些日志框架也利用ThreadLocal来存储与当前线程相关的日志上下文信息, 如用户ID、操作类型等。这样, 在记录日志时, 可以方便地获取这些信息, 并将其添加到日志条目中, 从而方便后续的日志分析和排查问题。

5. RPC

- 在远程过程调用(RPC)框架中, ThreadLocal用于存储和传递与当前调用相关的上下文信息。这些上下文信息可能包括调用者的身份、调用的参数、超时设置等。通过将这些信息存储在ThreadLocal中, 可以确保它们在RPC调用过程中能够被正确地传递和使用。

6. Hibernate

- SessionContext:** 用于存储当前线程的Hibernate会话相关数据, 如当前会话、持久化上下文等。
- TransactionManager:** 管理事务状态, 每个线程可以有独立的事务状态, 如当前是否在事务中。

7. Tomcat

- ThreadLocal变量:** 用于跟踪每个请求的会话信息、用户认证数据等, 确保这些数据不会在请求之间共享。

8. Kafka

- Producer and Consumer Threads:** 在消息生产和消费过程中, 使用ThreadLocal来存储线程特定的配置和状态信息。

ThreadLocal综合使用案例

由于ThreadLocal使用不当会导致严重的内存泄漏, 所以为了更好的避免内存泄漏的发生, 我们使用ThreadLocal时遵守以下两个原则:

- 尽量使用private static final修饰ThreadLocal实例。使用 private 与final 修饰符, 主要是尽可能不让他人修改、变更ThreadLocal变量的引用; 使用static 修饰符主要为了确保ThreadLocal实例的全局唯一。
- ThreadLocal使用完成之后务必调用remove方法。这是简单、有效地避免ThreadLocal引发内存泄漏的方法。

下面用一个综合案例演示一下ThreadLocal的使用。此案例的功能为：记录执行过程中所调用的函数的执行耗时。比如在实际Web开发过程中，一次客户端请求往往会涉及到DB、缓存、RPC等多个耗时调用，一旦出现性能问题，就需要记录一下各个点耗时的时间，从而判断性能的瓶颈所在。下面的代码定义了三个方法 `serviceMethod`、`daoMethod`、`rpcMethod`，用于模拟实际的DB、RPC等耗时调用，具体的代码如下：

```
package com.crazymakercircle.mutithread.basic.threadlocal;
...省略import
public class ThreadLocalTest2
{
    /**
     * 模拟业务方法
     */
    public void serviceMethod()
    {
        //睡眠500ms,模拟执行耗时
        sleepMilliseconds(500);

        //记录从开始调用到当前这个点（"point-1"）的耗时
        SpeedLog.logPoint("point-1 service");

        //调用DAO方法：模拟dao业务方法
        daoMethod();

        //调用RPC方法：模拟RPC远程业务方法
        rpcMethod();
    }

    /**
     * 模拟dao业务方法
     */
    public void daoMethod()
    {
        //睡眠400ms,模拟执行耗时
        sleepMilliseconds(400);

        //记录上一个点（"point-1"）这里（"point-2"）的耗时
        SpeedLog.logPoint("point-2 dao");
    }

    /**
     * 模拟RPC远程业务方法
     */
    public void rpcMethod()
    {
        //睡眠400ms,模拟执行耗时
        sleepMilliseconds(600);

        //记录上一个点（"point-2"）这里（"point-3"）的耗时
        SpeedLog.logPoint("point-3 rpc");
    }
    ...省略不相关代码
}
```

为了能灵活地记录各个执行埋点的耗时，这里定义了一个SpeedLog类。该类含有一个ThreadLocal类型的、初始值为一个Map<String, Long>实例的“线程本地变量”，名字叫做TIME_RECORD_LOCAL。

如果要记录某个函数的调用耗时，就需要进行耗时埋点，具体的方法为logPoint(String point)。该方法会操作TIME_RECORD_LOCAL本地变量，在其中增加一次耗时记录：Key为耗时埋点的名称，值为当前时间和上一次记录时间的差值，也就是上一次埋点到本次埋点之间的调用耗时。

SpeedLog类的代码，大致如下：

```
package com.crazymakercircle.mutithread.basic.threadlocal;
...省略import
public class SpeedLog
{
    /**
     * 记录调用耗时的本地Map变量
     */
    private static final ThreadLocal<Map<String, Long>>
        TIME_RECORD_LOCAL =ThreadLocal.withInitial(SpeedLog::initialStartTime);

    /**
     * 记录调用耗时的本地Map变量的初始化方法
     */
    public static Map<String, Long> initialStartTime()
    {
        Map<String, Long> map = new HashMap<>();
        map.put("start", System.currentTimeMillis());
        map.put("last", System.currentTimeMillis());
        return map;
    }

    /**
     * 开始耗时记录
     */
    public static final void beginSpeedLog()
    {
        Print.fo("开始耗时记录");
        TIME_RECORD_LOCAL.get();
    }

    /**
     * 结束耗时记录
     */
    public static final void endSpeedLog()
    {
        TIME_RECORD_LOCAL.remove();
        Print.fo("结束耗时记录");
    }

    /**
     * 耗时埋点
     */
    public static final void logPoint(String point)
    {
        //获取上一次的时间
        Long last = TIME_RECORD_LOCAL.get().get("last");
        //计算上一次埋点到当前埋点的耗时
        Long cost = System.currentTimeMillis() - last;

        //保存上一次埋点到当前埋点的耗时
    }
}
```

```

        TIME_RECORD_LOCAL.get().put(point + " cost:", cost);

        //保存当前时间，供下一次埋点使用
        TIME_RECORD_LOCAL.get().put("last", System.currentTimeMillis());
    }
    ...省略不相干代码
}

```

下面是一个测试用例，演示一下在 serviceMethod、daoMethod、rpcMethod三个模拟方法的调用过程中，其耗时的记录和输出，具体的代码如下：

```

package com.crazymakercircle.mutithread.basic.threadlocal;
...省略import
public class ThreadLocalTest2
{

    /**
     * 测试用例：线程方法调用的耗时
     */
    @org.junit.Test
    public void testSpeedLog() throws InterruptedException
    {
        Runnable runnable = () ->
        {
            //开始耗时记录，保存当前时间
            SpeedLog.beginSpeedLog();
            //调用模拟业务方法
            serviceMethod();
            //打印耗时
            SpeedLog.printCost();
            //结束耗时记录
            SpeedLog.endSpeedLog();

        };
        new Thread(runnable).start();
        sleepSeconds(10); //等待10s看结果
    }
    ...省略不相干代码
}

```

运行以上用例，三个模拟方法 serviceMethod、daoMethod、rpcMethod的耗时输出如下：

```

[SpeedLog.beginSpeedLog]: 开始耗时记录
[SpeedLog.printCost]: start =>1600347227334
[SpeedLog.printCost]: point-1 service cost: =>500
[SpeedLog.printCost]: point-2 dao cost: =>401
[SpeedLog.printCost]: point-3 rpc cost: =>600
[SpeedLog.printCost]: last =>1600347228835
[SpeedLog.endSpeedLog]: 结束耗时记录

```

以上案例中，将ThreadLocal变量声明成为 private static final 的形式，使得外部不能直接访问，外部能访问的是将ThreadLocal变量封装之后的接口函数如beginSpeedLog()、logPoint(String point)、endSpeedLog()等等。

总之，使用ThreadLocal能实现每个线程都有一份变量的本地值，其原因是由于每个线程都有自己独

立的ThreadLocalMap空间，本质上属于以空间换时间的设计思路，该设计思路属于了另一种意义的“无锁编程”。

ThreadLocal 使用总结

总结起来，ThreadLocal在实际项目中的应用广泛且实用，但同时也需注意其潜在的风险和挑战。通过遵循最佳实践，合理设计和管理ThreadLocal变量，我们能够在多线程环境下高效地解决数据隔离问题，同时保持代码简洁易懂，确保系统稳定性和高性能运行。

ThreadLocal的实现原理

在早期的JDK版本中，ThreadLocal的内部结构是一个Map，其中每一个线程实例作为Key，线程在“线程本地变量”中绑定的值为Value（本地值）。早期版本中的Map结构，其拥有者为ThreadLocal，每一个ThreadLocal实例，拥有一个Map实例。

在JDK8版本中，ThreadLocal的内部结构发生了演进，虽然还是使用了Map结构，但是Map结构的拥有者已经发生了变化，其拥有者为Thread（线程）实例，每一个Thread实例，拥有一个Map实例。另外，Map结构的Key值也发生了变化：新的Key为ThreadLocal实例。

在JDK8版本中，每一个Thread线程内部都有一个Map(ThreadLocalMap)，因为如我们给一个Thread创建多个Threadlocal实例，然乎放置本地数据，那么当前线程的ThreadLocalMap中就会有多个“Key-Value对”，其中ThreadLocal实例为key，本地数据为Value。

在代码的层面来说，新版本的ThreadLocalMap还是由ThreadLocal类维护的，由ThreadLocal负责ThreadLocalMap实例的获取、创建，并从中设置本地值、获取本地值。所以ThreadLocalMap还寄存于ThreadLocal内部，而并没有被迁移到Thread内部。

ThreadLocal内部使用ThreadLocalMap来保存每个线程的变量副本。每个Thread都持有一个ThreadLocalMap的引用，这个map的key是ThreadLocal实例本身，value是线程变量副本。

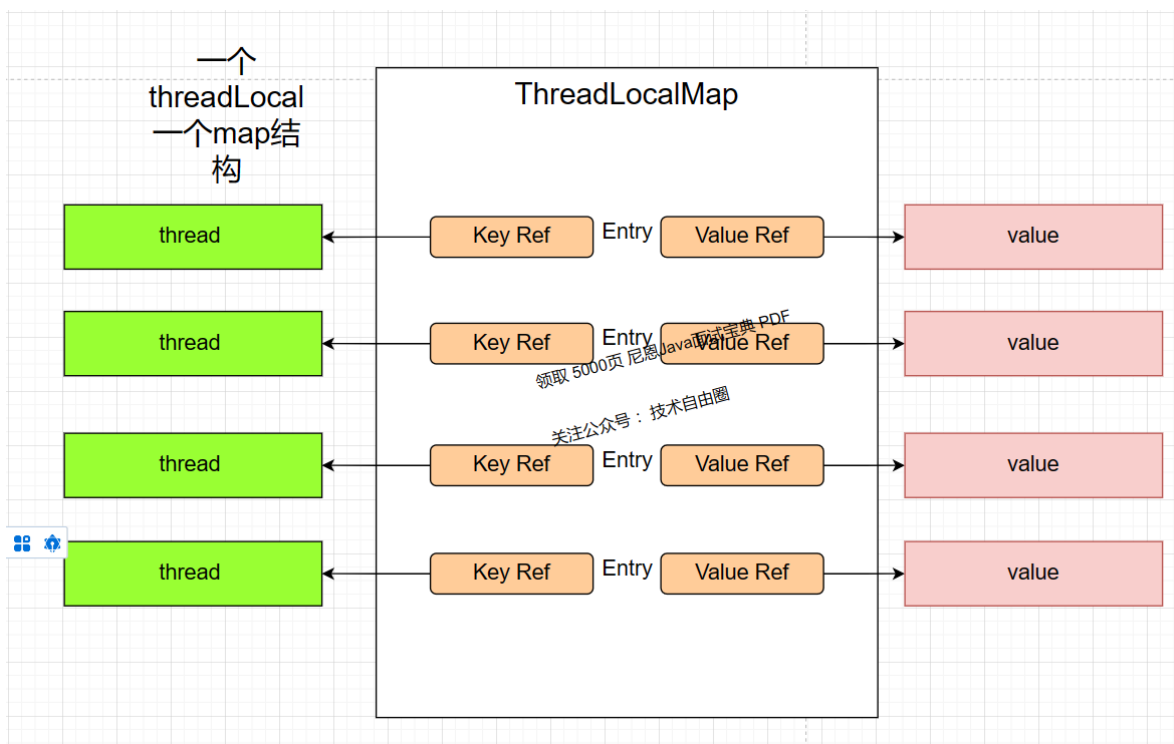
当线程首次调用ThreadLocal的get()或set()方法时，ThreadLocalMap会被创建并关联到当前线程。此后，线程就可以通过ThreadLocal实例的get()和set()方法访问自己的线程局部变量了。

简单来说：ThreadLocal 就是一种以 空间换时间 的做法，在每个 Thread 里面维护了一个以开放定址法实现的ThreadLocal.ThreadLocalMap，把数据进行隔离，数据不共享，自然就没有线程安全方面的问题了。

ThreadLocal内部结构演进

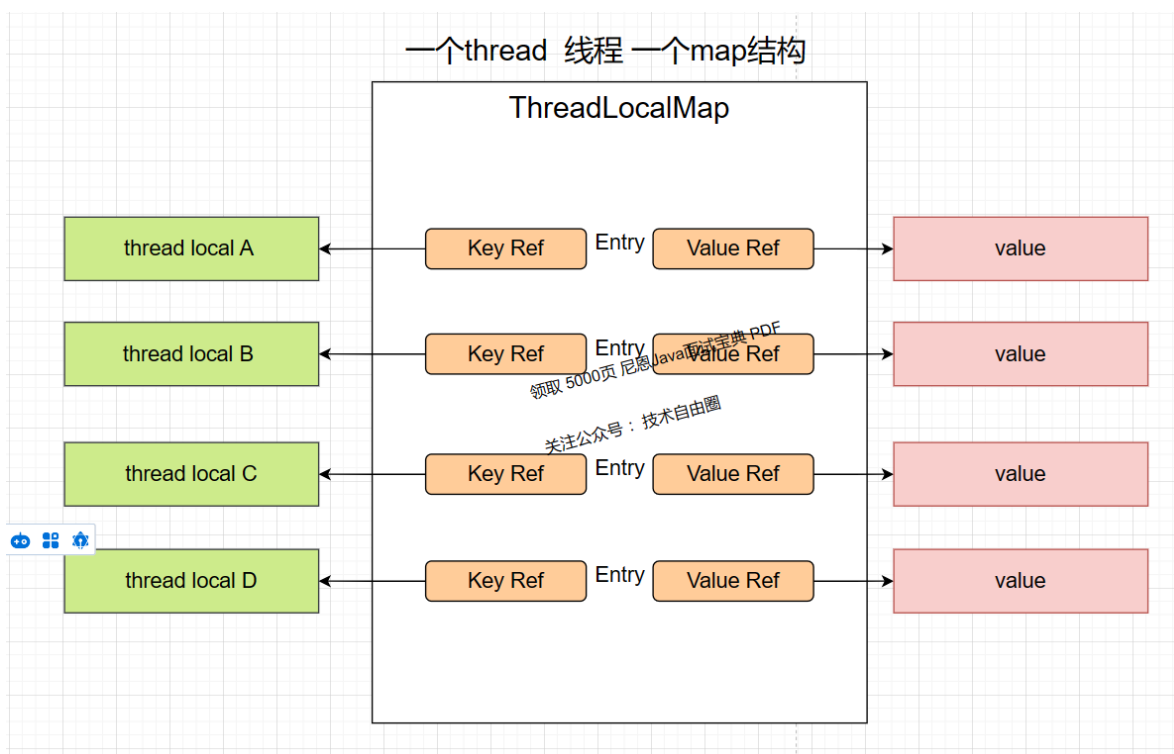
在早期的JDK版本中，ThreadLocal的内部结构是一个Map，其中每一个线程实例作为Key，线程在“线程本地变量”中绑定的值为Value（本地值）。早期版本中的Map结构，其拥有者为 ThreadLocal，每一个ThreadLocal实例，拥有一个 Map 实例。

JDK1.7的ThreadLocalMap 内部结构，大致如下：



在JDK8版本中，ThreadLocal的内部结构发生了演进，虽然还是使用了Map结构，但是Map结构的拥有者已经发生了变化，其拥有者为 Thread（线程）实例，每一个Thread实例，拥有一个 Map 实例。另外，Map结构的Key值也发生了变化：新的 Key为ThreadLocal实例。

JDK1.8的ThreadLocalMap 内部结构，大致如下：



在JDK8版本中，每一个Thread线程内部都有一个Map(ThreadLocalMap)，因为如我们给一个Thread创建多个Threadlocal实例，然后放置本地数据，那么当前线程的ThreadLocalMap中就会有多个“Key-Value对”，其中 ThreadLocal实例为key，本地数据为Value。

在代码的层面来说，新版本的ThreadLocalMap还是由ThreadLocal类维护的，由ThreadLocal负责ThreadLocalMap实例的获取、创建，并从中设置本地值、获取本地值。

所以ThreadLocalMap还寄存于ThreadLocal内部，而并没有被迁移到Thread内部。

每一个线程在获取本地值时，将ThreadLocal实例作为Key从自己拥有的ThreadLocalMap中获取值，别的线程无法访问自己的ThreadLocalMap实例，自己也无法访问别人ThreadLocalMap实例，达到相互隔离，互不干扰。

jdk1.8版本ThreadLocalMap 与早期版本的ThreadLocalMap实现相比，主要的变化为：

- **拥有者发生了变化**：新版本的ThreadLocalMap拥有者为 Thread ，早期版本的ThreadLocalMap拥有者为 ThreadLocal 。
- **Key发生了变化**：新版本的Key为 ThreadLocal 实例，早期版本的Key为 Thread 实例。

jdk1.8版本ThreadLocalMap 与早期版本的ThreadLocalMap实现相比，新版本的主要优势为：

(1) **每个ThreadLocalMap存储的“Key-Value对”数量变少。**

- 早期版本的“Key-Value对”数量与线程个数强关联，如果线程数量多，则ThreadLocalMap存储“Key-Value对”数量也多。
- 新版本的ThreadLocalMap的Key 为ThreadLocal实例，多线程情况下ThreadLocal实例比线程数少。

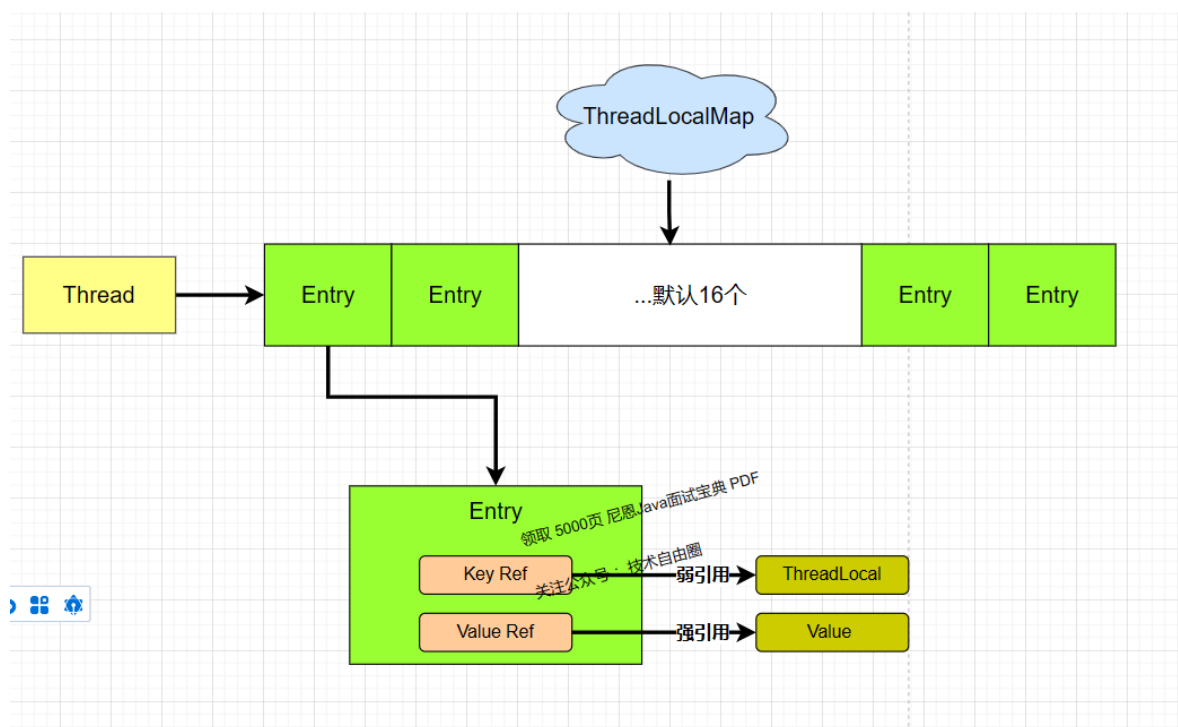
(2) **ThreadLocalMap拥有者为Thread，降低内存消耗。**

- 早期版本ThreadLocalMap的拥有者为ThreadLocal，在Thread（线程）实例销毁后，ThreadLocalMap还是存在的；
- 新版本的ThreadLocalMap的拥有者为Thread，现在当Thread实例销毁后，ThreadLocalMap也会随之销毁，在一定程度上能减少内存的消耗。

ThreadLocalMap对象和Entry是什么

ThreadLocal的操作都是基于 ThreadLocalMap 展开的，而 ThreadLocalMap 是ThreadLocal的一个静态内部类，其实现了一套简单的Map结构（比HashMap简单）。

ThreadLocalMap内部维护Entry数组，作为ThreadLocalMap条目数组，作为散列表使用，如下图：



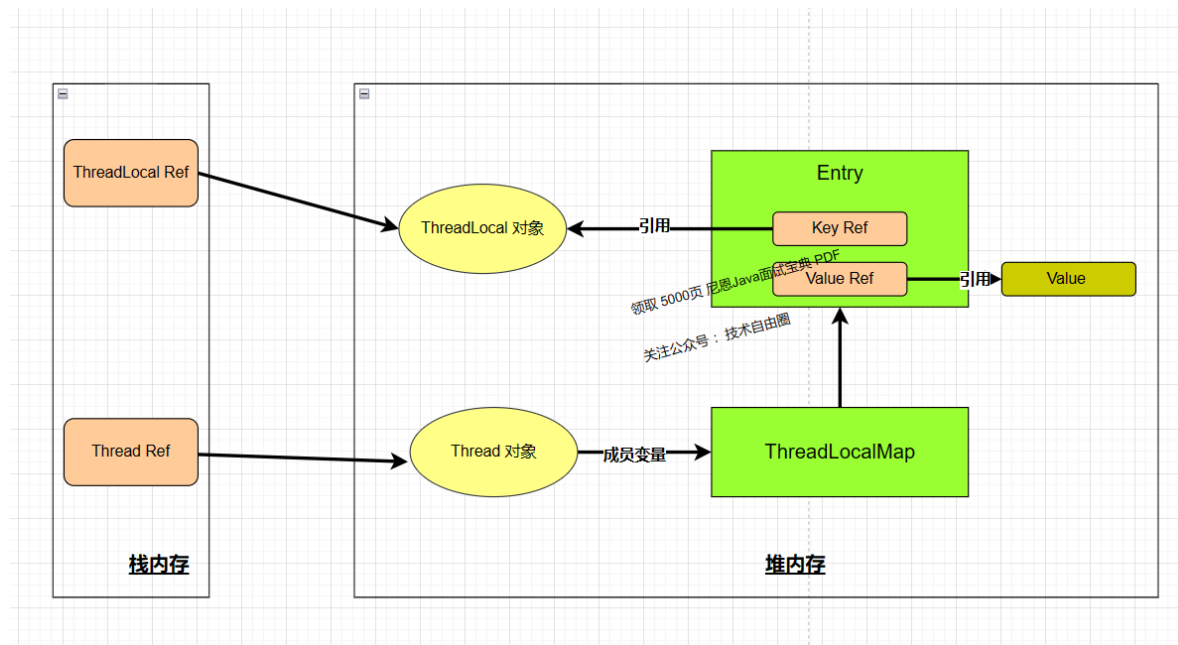
ThreadLocal的结构模型

以jdk1.8的ThreadLocal 为标准， 总结一下 ThreadLocal的结构模型。

每个Thread都有一个 ThreadLocalMap 结构， 其中就保存着当前线程所持有的所有 ThreadLocal。

ThreadLocal 本身只是一个引用， 没有直接保存值， 值是保存在 ThreadLocalMap 中， ThreadLocal 作为 key， 值（实际保存的数据）作为 value。

可以用下面的图来概括：



ThreadLocal源码分析

ThreadLocal的源码提供的方法不多，主要的方法有：`set(T value)`方法、`get()`方法、`remove()`方法、`initialValue()`方法。

set(T value)方法

`set(T value)`方法 用于设置“线程本地变量”在当前线程的ThreadLocalMap中对应的值，相当于设置线程本地值，其核心源码如下：

```
public void set(T value) {  
    //获取当前线程对象  
    Thread t = Thread.currentThread();  
  
    //获取当前线程的ThreadLocalMap 成员  
    ThreadLocalMap map = getMap(t);  
  
    //判断map是否存在  
    if (map != null)  
    {  
        //value被绑定到threadLocal实例  
        map.set(this, value);  
    }  
    else  
    {  
        // 如果当前线程没有ThreadLocalMap成员实例  
        // 创建一个ThreadLocalMap实例，然后，作为成员关联到t（thread实例）  
    }  
}
```

```

        createMap(t, value);
    }
}

// 获取线程t的ThreadLocalMap成员
ThreadLocalMap getMap(Thread t) {
    return t.threadLocals;
}

// 线程t创建一个ThreadLocalMap成员
// 并为新的Map成员设置第一个Key-value对，Key为当前的ThreadLocal实例
void createMap(Thread t, T firstValue) {
    t.threadLocals = new ThreadLocalMap(this, firstValue);
}

```

通过以上的源码，可以看出 `set(T value)` 方法的执行流程，大致如下：

- 获得当前线程，然后获得当前线程的ThreadLocalMap成员，暂存于map变量。
- 如果map不为空，则将Value设置到map中，当前的Threadlocal作为key。
- 如果map为空，给该线程创建map，然后设置第一个“Key-Value对”，Key为当前的ThreadLocal实例，Value为set方法的参数value值。

get()方法

`get()` 方法用于获取“线程本地变量”在当前线程的 `ThreadLocalMap` 中对应的值，相当于获取线程本地值，其核心源码如下：

```

public T get() {
    //获得当前线程对象
    Thread t = Thread.currentThread();
    //获得线程对象的ThreadLocalMap 内部成员
    ThreadLocalMap map = getMap(t);

    // 如果当前线程的内部map成员存在
    if (map != null) {
        // 以当前threadlocal为key,尝试获得条目
        ThreadLocalMap.Entry e = map.getEntry(this);
        // 条目存在
        if (e != null) {
            T result = (T)e.value;
            return result;
        }
    }
    // 如果当前线程对应map不存在
    //或者map存在，但是当前threadlocal实例没有对应的Key-value，返回初始值
    return setInitialValue();
}

// 设置threadlocal关联的初始值并返回
private T setInitialValue() {
    //调用初始化钩子函数，获取初始值
    T value = initialValue();
    Thread t = Thread.currentThread();
    ThreadLocalMap map = getMap(t);
    if (map != null)
        map.set(this, value);
    else

```

```

        createMap(t, value);
    return value;
}

```

通过以上的源码，可以看出 `T get()` 方法的执行流程，大致如下：

- 先尝试获得当前线程，然后获得当前线程的ThreadLocalMap成员，暂存于map变量。
- 如果获得的map不为空，以当前threadlocal实例为Key尝试获得map中的Entry（条目）。
- 如果Entry条目不为空，返回Entry中的Value。
- 如果Entry为空，则通过调用initialValue初始化钩子函数获取“ThreadLocal”初始值，并设置在map中。如果map不存在，还会给当前线程创建新ThreadLocalMap成员，并绑定第一个“Key-Value对”。

remove()方法

`remove()` 方法用于在当前线程的ThreadLocalMap中，移除“线程本地变量”所对应的值，其核心源码如下：

```

public void remove() {
    ThreadLocalMap m = getMap(Thread.currentThread());
    if (m != null)
        m.remove(this);
}

```

initialValue() 方法

当“线程本地变量”在当前线程的 ThreadLocalMap 中尚未绑定值时，`initialValue()` 方法用于获取初始值。其源码如下：

```

protected T initialValue() {
    return null;
}

```

如果没有调用set直接调用get，则会调用此方法，但是该方法只会被调用一次。

默认情况下，`initialValue()` 方法返回null，如果不想返回null，可以继承ThreadLocal覆盖此方法。

真的需要继承ThreadLocal去重写initialValue()方法吗？其实没有必要。

JDK已经为大家定义了一个ThreadLocal的内部SuppliedThreadLocal静态子类，并且提供了 `ThreadLocal.withInitial(...)` 静态工厂方法，方便大家在定义ThreadLocal实例时设置初始值回调函数。

使用工厂方法构造ThreadLocal实例的代码如下：

```

ThreadLocal<Foo> LOCAL_FOO = ThreadLocal.withInitial(() -> new Foo());

```

JDK定义的 `ThreadLocal.withInitial(...)` 静态工厂方法，以及其内部子类SuppliedThreadLocal的源码如下：

```

//ThreadLocal工厂方法，可以设置本地变量初始值钩子函数
public static <S> ThreadLocal<S> withInitial(
    Supplier<? extends S> supplier) {
    return new SuppliedThreadLocal<>(supplier);
}

```

```

//内部静态子类
//继承了ThreadLocal，重写了initialValue()方法，返回钩子函数的值作为初始值
static final class SuppliedThreadLocal<T> extends ThreadLocal<T> {
    //保存钩子函数
    private final Supplier<? extends T> supplier;
    //传入钩子函数
    SuppliedThreadLocal(Supplier<? extends T> supplier) {
        this.supplier = Objects.requireNonNull(supplier);
    }

    @Override
    protected T initialValue() {
        return supplier.get(); //返回钩子函数的值作为初始值
    }
}

```

ThreadLocalMap源码分析

ThreadLocal的操作都是基于 ThreadLocalMap 展开的，而 ThreadLocalMap 是ThreadLocal的一个静态内部类，其实现了一套简单的Map结构（比HashMap简单）。

ThreadLocalMap的主要成员变量

ThreadLocalMap 的成员变量与HashMap的成员变量非常类似，其内部的主要成员如下所示：

```

public class ThreadLocal<T> {
    ...省略其他
    static class ThreadLocalMap {
        //Map的条目数组，作为散列表使用
        private Entry[] table;
        //Map的条目初始容量16
        private static final int INITIAL_CAPACITY = 16;
        //Map的条目数量
        private int size = 0;
        //扩容因子
        private int threshold;

        //Map的条目类型，一个静态的内部类
        // Entry 继承子WeakReference,Key为ThreadLocal实例
        static class Entry extends WeakReference<ThreadLocal<?>> {
            Object value; //条目的值
            Entry(ThreadLocal<?> k, Object v) {
                super(k);
                value = v;
            }
        }
        ...省略其他
    }
}

```

ThreadLocal源码中 `get()`、`set()`、`remove()` 方法 都涉及到ThreadLocalMap的方法调用，主要调用了ThreadLocalMap的如下几个函数：

- `set(ThreadLocal<?> key, Object value)`：向Map实例设置“Key-Value对”。
- `getEntry(ThreadLocal)`：从Map实例获取Key（ThreadLocal实例）所属的Entry。
- `remove(ThreadLocal)`：根据Key（ThreadLocal实例）从Map实例移除所属的Entry。

作为参考，这里只将 ThreadLocalMap 的 set(ThreadLocal<?> key, Object value) 方法的代码以注释的形式做一个简单的分析，具体如下：

```
private void set(ThreadLocal<?> key, Object value) {
    Entry[] tab = table;
    int len = tab.length;

    //根据key的HashCode，找到key在数组上的槽点i
    int i = key.threadLocalHashCode & (len-1);

    // 从槽点i开始向后循环搜索，找空余槽点（空余位置）或者找现有槽点
    //如果没有现有槽点，则必定有空余槽点，因为没有空间时会扩容
    for (Entry e = tab[i]; e != null;
         e = tab[i = nextIndex(i, len)]) {
        ThreadLocal<?> k = e.get();
        //找到现有槽点：key值为ThreadLocal实例
        if (k == key) {
            e.value = value;
            return;
        }
        //找到异常槽点：槽点被GC掉，重设key值和value值
        if (k == null) {
            replaceStaleEntry(key, value, i);
            return;
        }
    }
    //没有找到现有的槽点，增加新的Entry
    tab[i] = new Entry(key, value);
    //设置ThreadLocal数量
    int sz = ++size;

    //清理key为null的无效Entry
    //没有可清理的Entry，并且现有条目数量大于扩容因子值，进行扩容
    if (!cleanSomeSlots(i, sz) && sz >= threshold)
        rehash();
}
```

Entry的Key需要使用弱引用

Entry 用于保存 ThreadLocalMap 的“key-value”条目，但是 Entry 使用了对 ThreadLocal 实例进行包装之后的弱引用（WeakReference）作为 Key，其代码如下：

```
// Entry 继承了WeakReference，并使用WeakReference对Key进行包装
static class Entry extends WeakReference<ThreadLocal<?>> {
    Object value; //值
    Entry(ThreadLocal<?> k, Object v) {
        super(k); //使用WeakReference对key值进行包装
        value = v;
    }
}
```

为什么 Entry 需要使用弱引用对 Key 进行包装，而不是直接使用 ThreadLocal 实例作为 Key 呢？

这个问题有点儿复杂，如果要分析清楚还有点难度。这里从一个简单的例子入手，假设有一个方法 funcA() 创建了一个“线程本地变量”，具体如下：

```

public void funcA()
{
    //创建一个线程本地变量
    ThreadLocal local = new ThreadLocal<Integer>();
    //设置值
    local.set(100);
    //获取值
    local.get();
    //函数末尾
}

```

当线程tn执行funcA方法到其末尾时，线程tn相关的JVM栈内存以及内部ThreadLocalMap成员的结构，大致如图1-20所示。

线程tn调用funcA（）方法新建了一个ThreadLocal实例，并使用local 局部变量指向这个实例，并且此local 是强引用；

在调用 local.set（100）之后，线程tn的ThreadLocalMap成员内部会新建一个Entry实例，其Key 以弱引用包装的方式指向ThreadLocal实例。

当线程tn执行完funcA方法后，funcA的方法栈帧将被销毁，强引用 local 的值也就没有了，但此时线程的ThreadLocalMap里的对应的Entry的 Key 引用还指向了 ThreadLocal 实例。

若Entry的 Key 引用是 强引用，就会导致Key引用指向的ThreadLocal实例、及其Value值都不能被GC回收，这将造成严重的 内存泄露，具体如图1-21所示。

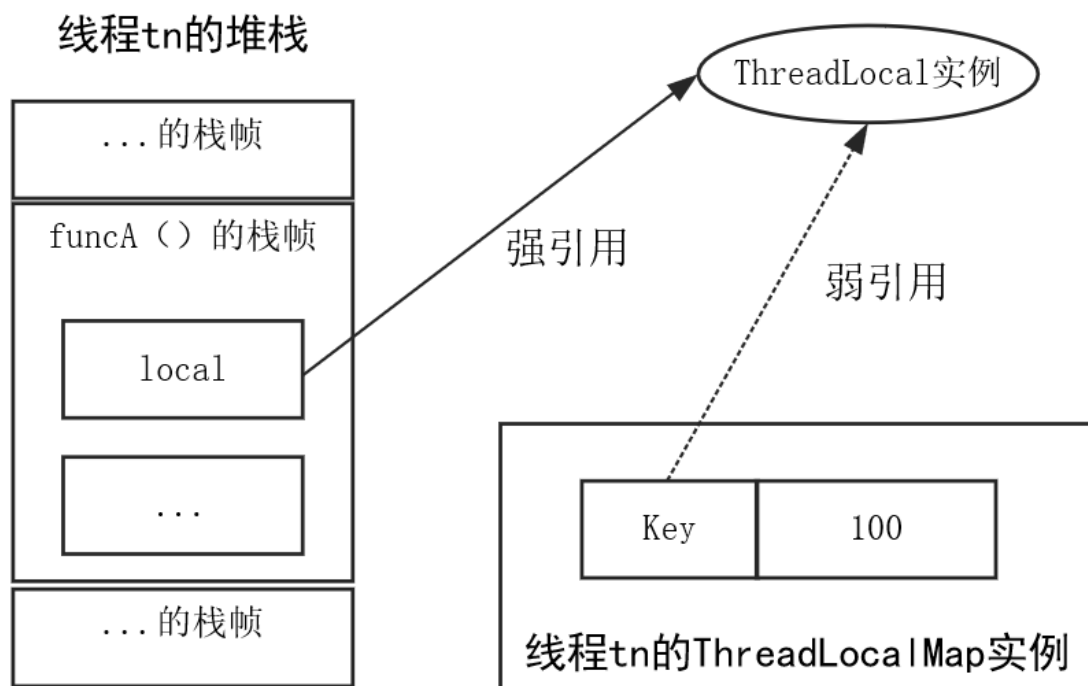


图1-20 当线程tn执行funcA方法末尾时内存结构

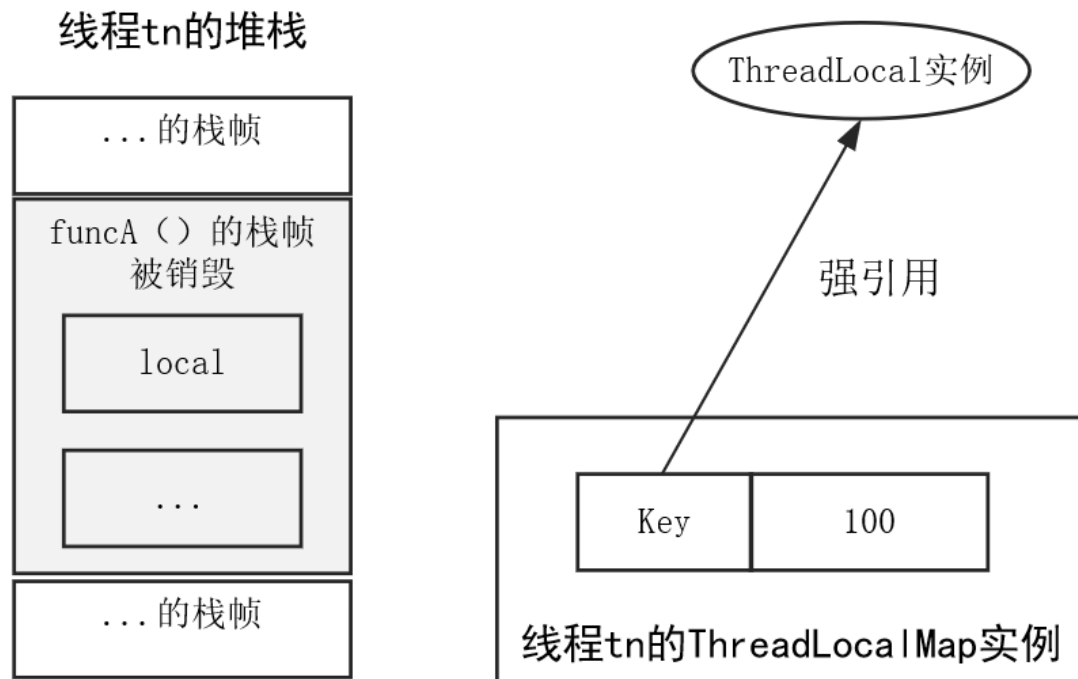


图1-21 若Entry的Key为强引用将导致ThreadLocal实例不能回收

什么是弱引用呢？

仅有弱引用（WeakReference）指向的对象，只能生存到下一次垃圾回收之前。

换句话说，当GC发生时，不管内存够不够，仅有弱引用所指向的对象都会被回收。而拥有强引用指向的对象，则不会被直接回收。

什么是内存泄漏呢？

不再用到的内存，没有及时释放，就叫做内存泄漏。

对于持续运行的服务进程，必须及时释放内存，否则内存占用率越来越高，轻则影响系统性能，重则导致进程崩溃。

由于ThreadLocalMap中Entry的Key使用了弱引用，在下次GC发生时，就可以使那些没有被其他强引用指向、仅被Entry的Key所指向的ThreadLocal实例能被顺利回收。并且，在Entry的Key引用被回收之后，其Entry的Key值变为null。后续当ThreadLocal的get、set或remove被调用时，ThreadLocalMap的内部代码会清除这些Key为null的Entry，从而完成相应的内存释放。

总结一下，使用ThreadLocal会发生内存泄漏的前提条件：

- 线程长时间运行而没有被销毁。线程池中的Thread实例很容易满足此条件。
- ThreadLocal引用被设置为null，且后续在同一Thread实例的执行期间，没有发生对其他ThreadLocal实例的get、set或remove操作。

只要存在一个针对任何ThreadLocal实例的get、set或remove操作，就会触发Thread实例拥有的ThreadLocalMap的Key为null的Entry清理工作，释放掉ThreadLocal弱引用为null的Entry。

综合以上两条可以看出：使用ThreadLocal出现内存泄漏还是比较容易的。但是一般公司对如何使用ThreadLocal都有编程规范要求，只要大家按照规范编写程序，也没有那么容易发生内存泄漏。

ThreadLocal造成内存泄露的问题

什么是内存泄漏？

不再用到的内存，没有及时释放，就叫做内存泄漏。

对于持续运行的服务进程，必须及时释放内存，否则内存占用率越来越高，轻则影响系统性能，重则导致进程崩溃。

ThreadLocal是怎么造成内存泄露的呢？

如果发生了下面的情况：

- 如果ThreadLocal是null了，也就是要被GC回收了，
- 但是此时我们的ThreadLocalMap（thread 的内部属性）生命周期和Thread的一样，它不会回收，这时候就出现了一个现象。

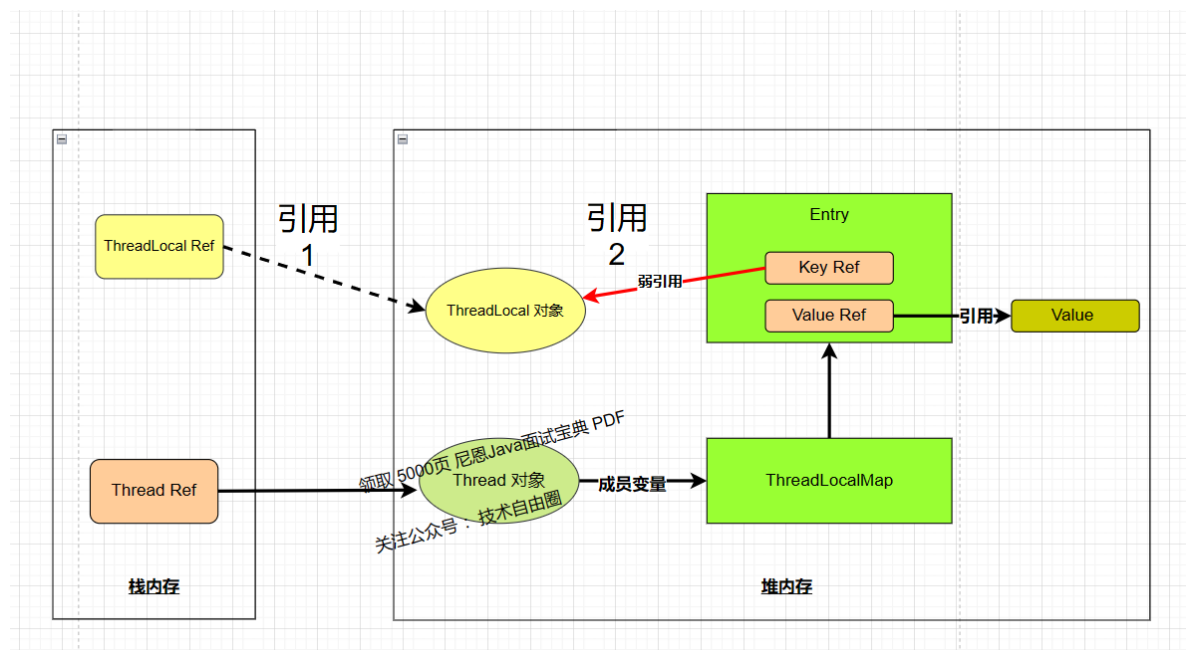
总之，就是ThreadLocalMap的key没了，但是value还在，这就造成了内存泄漏。

我们细致的分析一下。

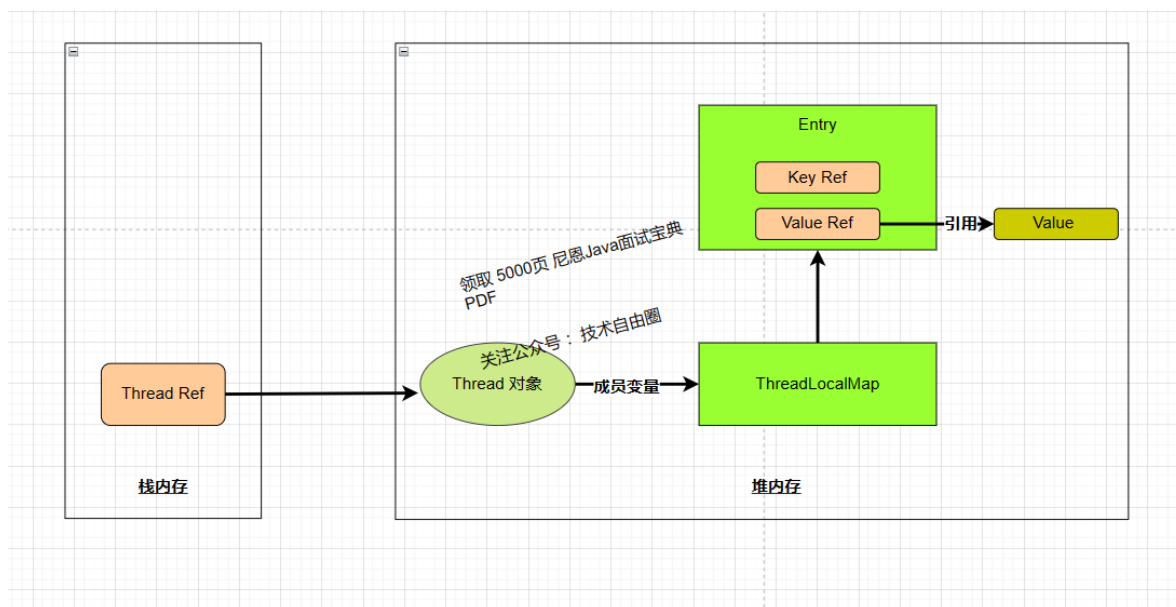
ThreadLocal 有两个引用链

ThreadLocalMap中的Key就是ThreadLocal对象，ThreadLocal 有两个引用链：

- 一个引用链是栈内存中ThreadLocal引用：
- 一个引用链是ThreadLocalMap中的Key对它的引用：



而对于Value（实际保存的值）来说，它的引用链只有一条，就是从Thread对象引用过来的，如下图：



上述过程分析后，就会出现如下的两种情况：

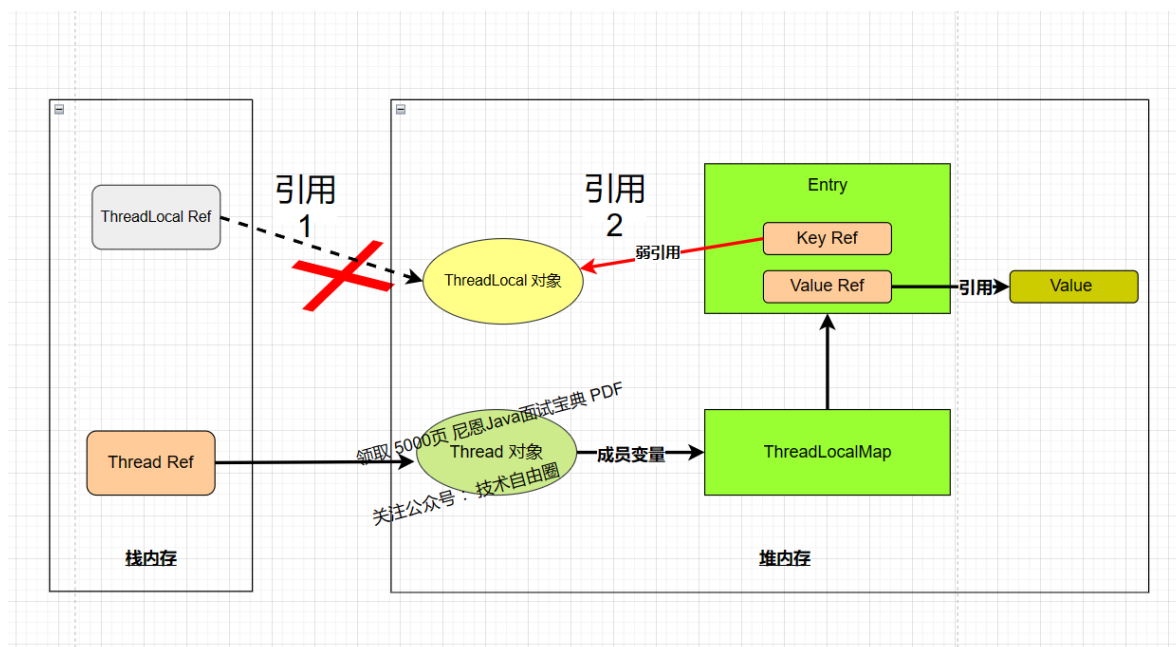
情况1：key的泄漏

情况2：value的泄漏

情况1：key的泄漏

栈上的ThreadLocal Ref引用不再使用了，即当前方法结束处理后，这个对象引用就不再使用了，

那么，ThreadLocal对象因为还有一条引用链存在，如果是强引用的话，这里就会导致ThreadLocal对象无法被回收，可能导致OOM。



情况1 的解决方案，使用弱引用解决。

情况2：value的泄漏

情况2.假设我们使用了线程池，如果Thread对象一直被占用使用中（如在线程池中被重复使用），但是此时我们的ThreadLocalMap（thread的内部属性）生命周期和Thread的一样，它不会回收，这时候就出现了一个现象。

这就意味着，Value这条引用链就一直存在，那么就会导致ThreadLocalMap无法被JVM回收，可能导致OOM，如上图。

情况2，比较严重。还得另想办法。

情况1的解决方案：使用弱引用，解决key的内存泄露

从如下ThreadLocal中内部类Entry代码可知：

Entry类的父类是弱引用WeakReference，ThreadLocal的引用k通过 WeakReference 构造方法传递给了 父类WeakReference的构造方法，

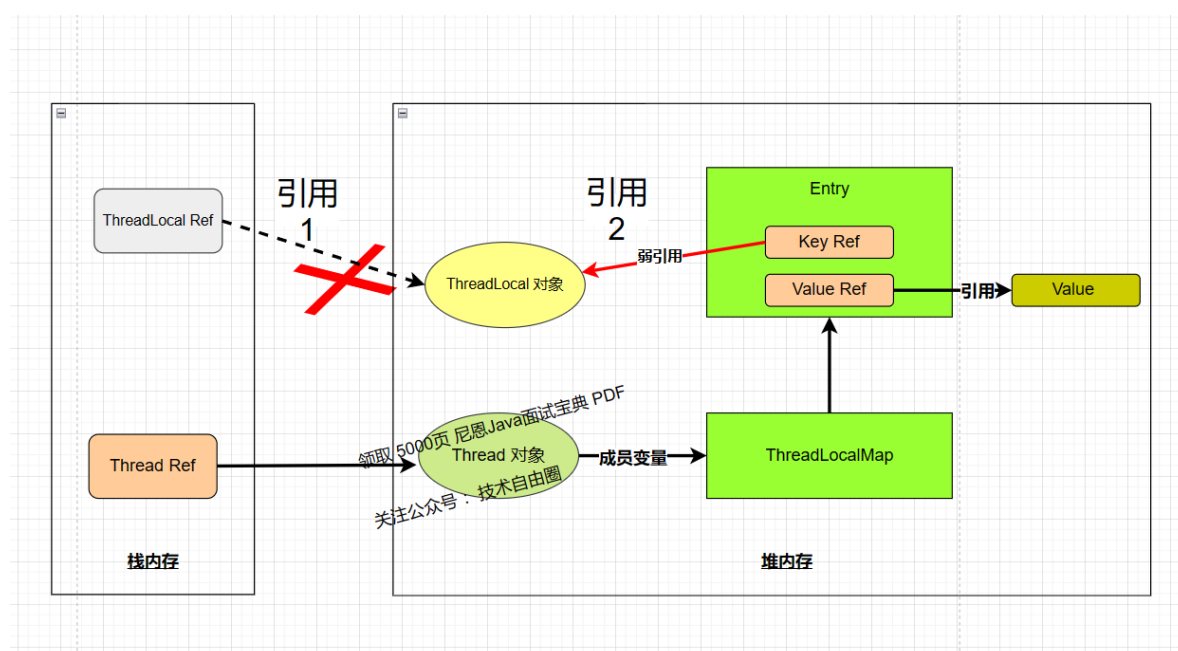
从而，ThreadLocalMap中的Key是ThreadLocal的弱引用，通过弱引用来解决内存泄露问题。

具体的代码如下

```
static class ThreadLocalMap {
    static class Entry extends WeakReference<ThreadLocal<?>> {
        Object value;
        Entry(ThreadLocal<?> k, Object v) {
            super(k); // key为弱引用
            value = v;
        }
    }
}
```

栈内存中的ThreadLocal Ref引用不再使用了，即当前方法结束处理后，这个key对象引用就不再使用了，

那么，如果这里 不用弱引用而是强引用的话，这里ThreadLocal对象因为还有一条引用链存在，所以就会导致他无法被回收，可能导致OOM。



回顾Java中4种引用类型

1. 强引用 (Strong Reference) :

- 这是最常见的引用类型。一个对象具有强引用，垃圾收集器就不会回收它，即使系统内存空间不足。
- 示例：`Object obj = new Object();` 在这里，`obj` 就是 `new Object()` 的一个强引用。

2. 软引用 (Soft Reference) :

- 用来描述一些可能还有用但并非必需的对象。在系统将要发生内存溢出异常前，将会把这些对象列进回收范围之中进行第二次回收。如果这次回收还没有足够的内存，才会抛出内存溢出异常。
- 在Java中，软引用是用来实现内存敏感的高速缓存。
- 示例：使用 `java.lang.ref.SoftReference` 类可以创建软引用。

3. 弱引用 (Weak Reference) :

- 这里讨论ThreadLocalMap中Entry类的重点。
- 弱引用也是用来描述非必需对象的，它的强度比软引用更弱一些，被弱引用关联的对象只能生存到下一次垃圾收集发生之前。当垃圾收集器工作时，无论当前内存是否足够，都会回收被弱引用关联的对象。
- 在Java中，弱引用是用来描述那些非关键的数据，在Java里用 `java.lang.ref.WeakReference` 类来表示。
- 示例：使用 `java.lang.ref.WeakReference` 类可以创建弱引用。

4. 虚引用 (Phantom Reference) :

- 一个虚引用关联着的对象，在任何时候都可能被垃圾收集器回收，它不能单独用来获取被引用的对象。虚引用必须和引用队列(`ReferenceQueue`)联合使用。主要用来跟踪对象被垃圾回收的活动。
- 虚引用对于一般的应用程序来说意义不大，主要使用在能比较精确控制Java垃圾收集器的高级场景中。
- 示例：使用 `java.lang.ref.PhantomReference` 类可以创建虚引用。

弱引用也是用来描述非必需对象的，它的强度比软引用更弱一些，被弱引用关联的对象只能生存到下一次垃圾收集发生之前。当垃圾收集器工作时，无论当前内存是否足够，都会回收被弱引用关联的对象。

从而解决 key的泄漏问题。

至此，key的泄漏问题，JDK已经帮我们顺利解决。

更复杂的是：如何解决value内存泄露问题？

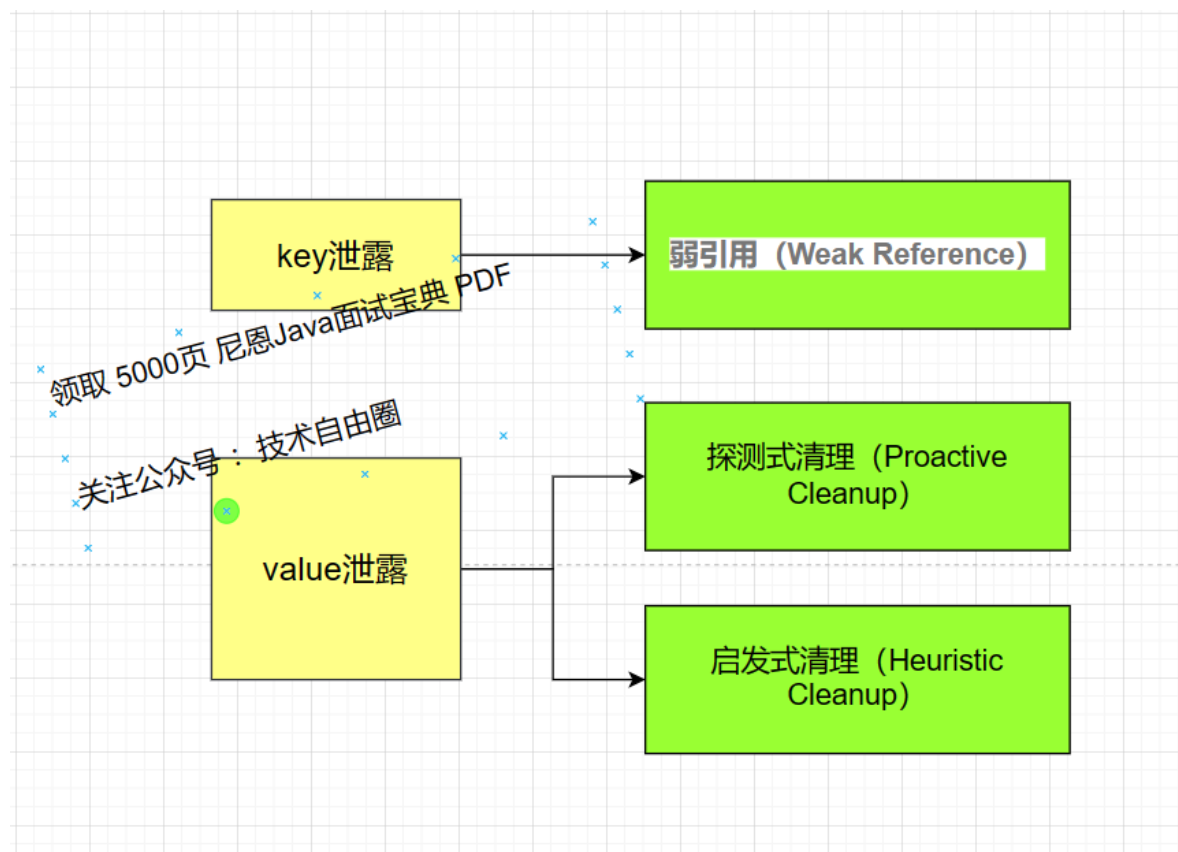
这个问题，网上有上万篇文章，都是抄来抄去，没有一篇文章写清楚了。

关于ThreadLocal内存泄漏，由于底层很复杂，导致尽管大家看了那些文章，甚至看了很多视频，还是不能理解透彻，迷迷糊糊的，面试的时候说不清楚，也很容易忘了。

尼恩团队看不下去，用咱们的雄厚技术实力（洪荒之力），给大家来一次穿透式的解读。

尼恩团队用这个《ThreadLocal学习圣经》，第一次帮助大家搞清楚ThreadLocal。

情况2的解决方案：清理策略解决value内存泄露



为了解决value内存泄露问题，Java 的 ThreadLocal 实现了两大清理方式：

- 探测式清理 (Proactive Cleanup)
- 启发式清理 (Heuristic Cleanup) 。

源码：value的 探测式清理：

当线程调用 ThreadLocal 的 get()、set() 或 remove() 方法时，会触发对 ThreadLocalMap 的清理。

此时，ThreadLocalMap 会检查所有键 (ThreadLocal 实例)，并移除那些已经被垃圾回收的key键及其对应的value 值。

这种清理是主动的，因为它是在每次操作 ThreadLocal 时进行的。

探测式清理 (Proactive Cleanup) 如何实现的呢？：

从当前节点开始遍历数组，将key等于null的entry置为null，key不等于null则rehash重新分配位置，若重新分配上的位置有元素则往后顺延。

注意：这里把清理的开销放到了get、set操作上，如果get的时候无用Entry (Entry的Key为null) 特别多，那这次get相对而言就比较慢了。

```
private int expungeStaleEntry(int staleSlot) {
    Entry[] tab = table;
    int len = tab.length;

    // expunge entry at staleSlot
    tab[staleSlot].value = null;
    tab[staleSlot] = null;
    size--;

    // Rehash until we encounter null
    Entry e;
```

```

int i;
for (i = nextIndex(staleSlot, len);
     (e = tab[i]) != null;
     i = nextIndex(i, len)) {
    ThreadLocal<?> k = e.get();
    if (k == null) {
        // 将k=null的entry置为null
        e.value = null;
        tab[i] = null;
        size--;
    } else {
        // k不为null, 则rehash从新分配配置
        int h = k.threadLocalHashCode & (len - 1);
        if (h != i) {
            tab[i] = null;

            // Unlike Knuth 6.4 Algorithm R, we must scan until
            // null because multiple entries could have been stale.
            while (tab[h] != null)
                // 重新分配后的位置上有元素则往后顺延。
                h = nextIndex(h, len);
            tab[h] = e;
        }
    }
}
return i;
}

```

源码：value的启发式清理：

在 `ThreadLocalMap` 的 `set()` 方法中，有一个阈值（默认为 `ThreadLocalMap.Entry` 数组长度的 1/4）。

当 `ThreadLocalMap` 中的 `Entry` 对象被删除（通过键的弱引用被垃圾回收）并且剩余的 `Entry` 数量大于这个阈值时，会触发一次启发式清理操作。

这种清理是启发式的，因为它不是每次操作都进行，而是基于一定的条件和概率。

启发式清理 (Heuristic Cleanup) 如何实现？：

从当前节点开始，进行do-while循环检查清理过期key，结束条件是连续n次未发现过期key就跳出循环，n是经过位运算计算得出的，可以简单理解为数组长度的2的多少次幂次。

```

private boolean cleansomeSlots(int i, int n) {
    boolean removed = false;
    Entry[] tab = table;
    int len = tab.length;
    do {
        i = nextIndex(i, len);
        Entry e = tab[i];
        if (e != null && e.get() == null) {
            n = len;
            removed = true;
            // 移除
            i = expungeStaleEntry(i);
        }
    } while ((n >>= 1) != 0);
    return removed;
}

```

```
}
```

业务主动清理：手动清除解决内存泄露

要知道，ThreadLocal的一个常见问题是内存泄露。

这通常发生在使用线程池的场景中，因为线程池中的线程通常是长期存在的，它们的ThreadLocal变量也不会自动清理，这可能导致内存泄漏。

前面讲了，JDK已经用尽全力去解决了，JDK用了三个办法，来解决内存泄漏。

尽管有弱引用以及这些清理机制，但最佳实践业务主动清理，

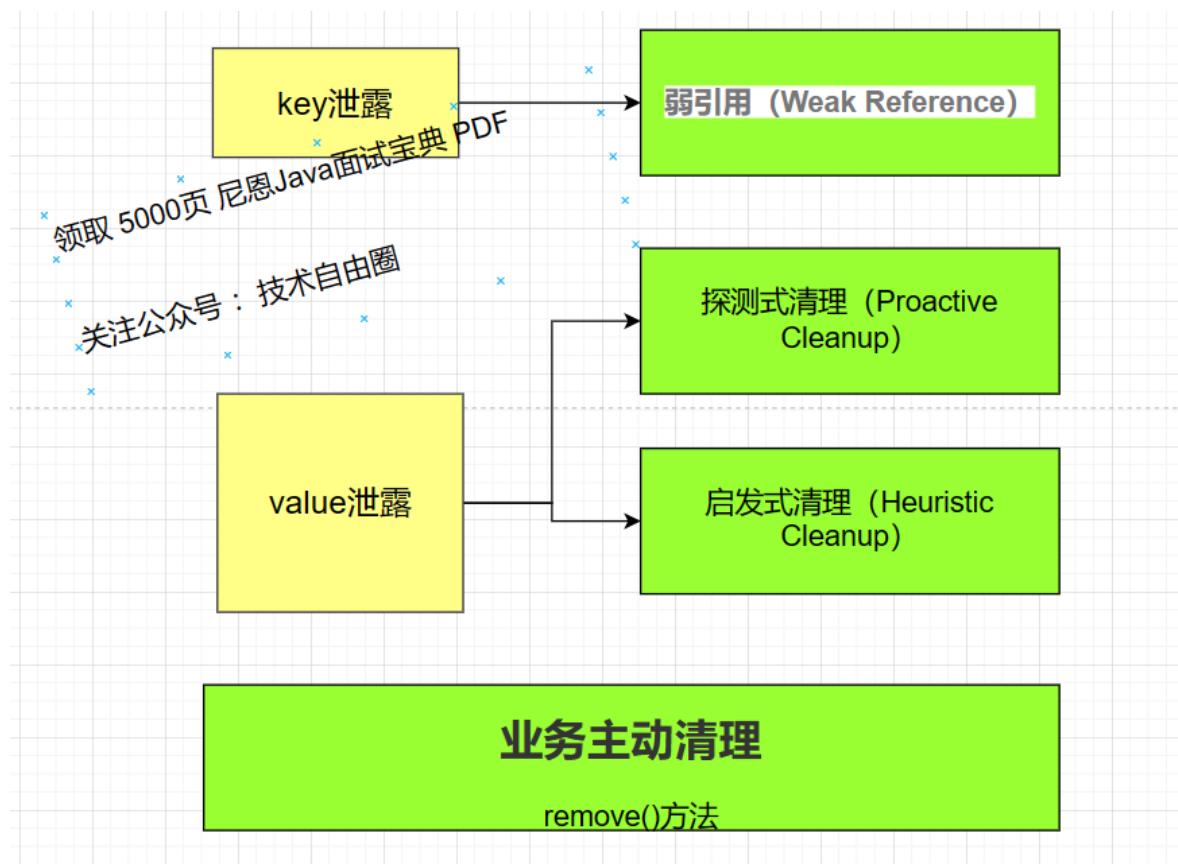
业务上解决这个问题的一个方法是，每当使用完ThreadLocal变量后，显式地调用 `remove()` 方法来清除它：

如何业务主动清理？在使用完 ThreadLocal 后显式调用 `remove()` 方法，以确保不再需要的值能够被及时回收，key和value 都同时清理，一锅端。

这样可以避免潜在的内存泄漏问题，并减少垃圾回收的压力。

大总结：ThreadLocal的内存泄露与解决方案

讲到这里，尼恩团队用一个大的图总结一下 ThreadLocal的内存泄露与解决方案，具体如下：



此图，是全网的第一张彻底穿透式的解读 ThreadLocal的内存泄露图。

用这个图，尼恩团队 第一次，帮助大家搞清楚复杂的 ThreadLocal的内存泄露。

此图很容易理解，很容易记忆。

大家可收藏起来，面试之前复习一下，一定能吊打面试官。**不对，是吊死面试官。**

尼恩团队用深厚的架构功力，非常喜欢也非常善于，把复杂的问题做清晰深入的穿透式、起底式的分析：

- 比如Netty的内存池和对象池（那个超级难，很多人穷其一生都搞不懂），
- 比如DDD的建模和落地，
- 比如Caffeine的底层架构，
- 比如高性能葵花宝典
- 等等等等。

这个技术难题一旦掌握，大家内力猛涨。所以，建议大家去看看尼恩的这些核心内容。

而且，尼恩团队进行一次真正的AI架构穿透，帮助大家穿透AI架构。

扯远了，言归正传。

ThreadLocal与内存泄露：防范与诊断

ThreadLocal的一个常见问题是内存泄露。

这通常发生在使用线程池的场景中，因为线程池中的线程通常是长期存在的，它们的ThreadLocal变量也不会自动清理，这可能导致内存泄漏。

JDK 用了三个办法，来解决内存泄漏。

业务上解决这个问题的一个方法是，每当使用完ThreadLocal变量后，显式地调用 `remove()` 方法来清除它：

使用ThreadLocal的性能问题和优化措施

虽然ThreadLocal提供了很方便的线程隔离机制，但有性能损耗的。

ThreadLocal的性能开销

ThreadLocal的性能开销主要来自两个方面：

- ThreadLocalMap的维护。
- ThreadLocal变量的创建和销毁。

在使用ThreadLocal时，尤其是在高开发的环境下，要注意其对性能的影响。

因此，在使用ThreadLocal时，要尽量复用、重用ThreadLocal变量，避免在高频率的操作中频繁地创建和销毁它们。`

编程规范：推荐使用 static final 修饰ThreadLocal对象

如何 要尽量复用、重用ThreadLocal变量？

编程规范有云：ThreadLocal 实例作为ThreadLocalMap的Key，针对一个线程内所有操作是共享的，所以建议设置static修饰符，以便被所有的对象共享。

由于静态变量会在类第一次被使用时装载，只会分配一次存储空间，此类的所有实例都会共享这个存储空间，所以使用 `static` 修饰ThreadLocal 就会节约内存空间。

另外，为了确保ThreadLocal实例的唯一性，除了使用static修饰之外，还会使用final进行加强修饰，以防止其在使用过程中发生动态变更。参考的实例如下：

```
//推荐使用static final线程本地变量
private static final ThreadLocal<Foo> LOCAL_FOO = new ThreadLocal<Foo>();
```

以上代码，为什么ThreadLocal实例除了添加static final 修饰之后，还常常加上了private修饰呢？

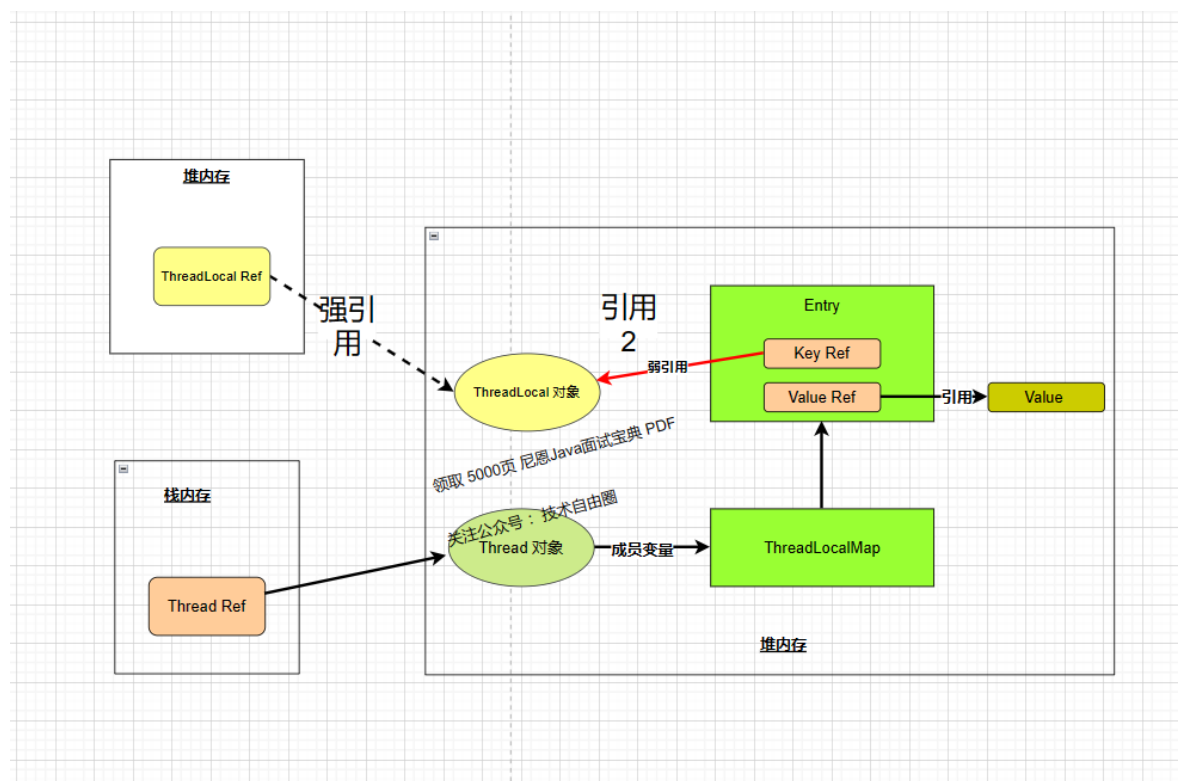
主要目的是 缩小使用的范围，尽可能不让他人引用。

凡事都有两面性，使用static、final修饰ThreadLocal实例也会带来副作用：内存泄漏。

- 为啥内存泄露又出来了？
- 上面不是解决了吗？
- 呜呜呜 !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

由于使用static、final修饰ThreadLocal对象实例，导致了咱们这个被 ThreadLocalMap中Entry的Key所引用的ThreadLocal对象实例，一直存在强引用。

这里有一个严重后果，这个 使用static、final修饰ThreadLocal对象实例 一直不会被GC，一直存在，一直存在。

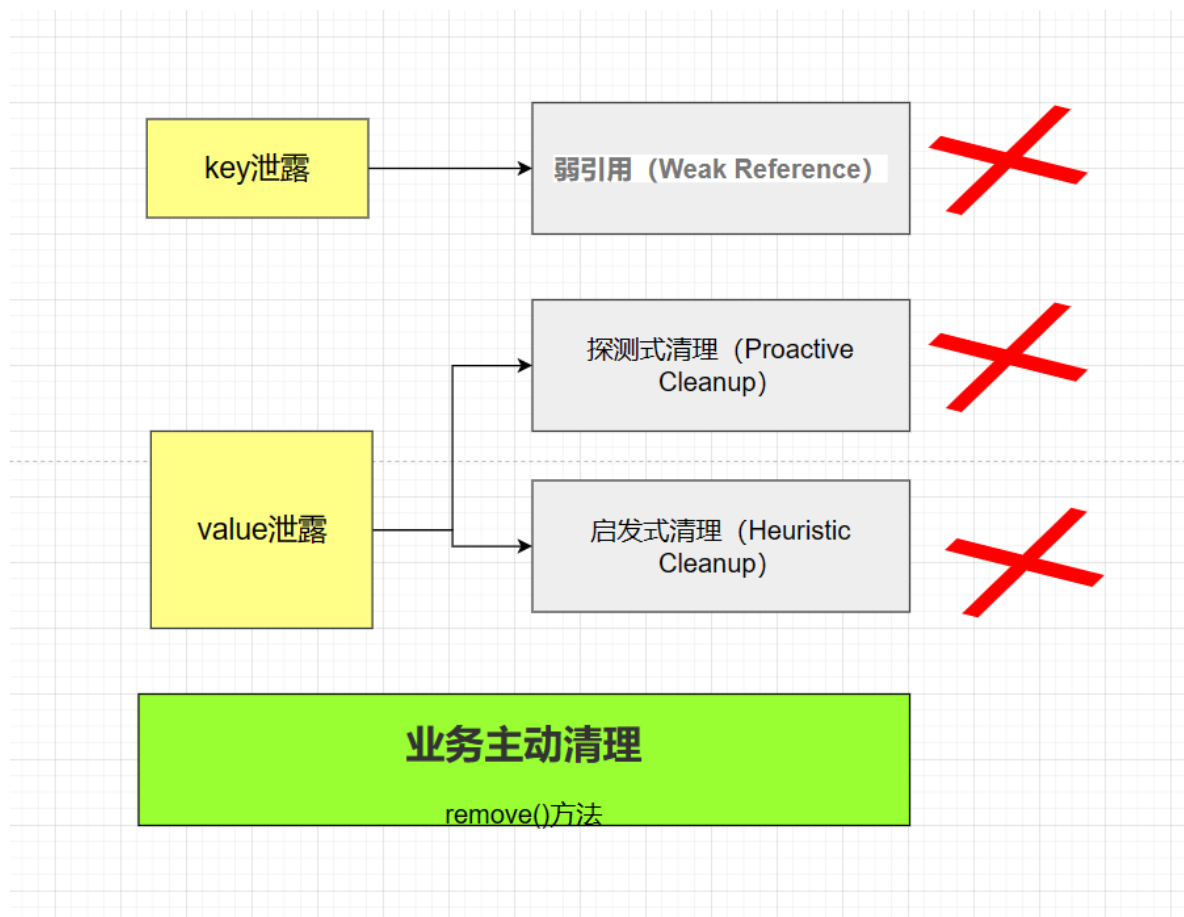


TheadLocal对象实例存在强引用，会导致三个彻底失效：

导致JDK解决key内存泄露问题的弱引用清理方式彻底失效。

导致JDK解决value内存泄露问题的两大清理方式彻底失效。

- 探测式清理 (Proactive Cleanup) 彻底失效
- 启发式清理 (Heuristic Cleanup) 彻底失效。



这使得Thread实例内部的ThreadLocalMap中Entry的Key在Thread实例的生命期内将始终保持为非null，从而导致Key所在的Entry不会被自动清空，这就会导致Entry中的Value指向的对象一直存在强引用，Value指向的对象在线程生命期内不会被释放，最终导致内存泄露。

所以，使用static、final修饰ThreadLocal实例，使用完后必须使用 `remove()` 进行手动释放。

如果使用线程池，可以定制线程池的 `afterExecute` 方法（任务执行完成之后的钩子方法），在任务执行完成之后，调用ThreadLocal实例的 `remove()` 方法 对其手动释放，从而实现的其线程内部的Entry得到释放，参考的代码如下：

```
//线程本地变量,用于记录线程异步任务的开始执行时间
private static final ThreadLocal<Long> START_TIME= new ThreadLocal<>();
ExecutorService pool = new ThreadPoolExecutor(2,
    4, 60,
    TimeUnit.SECONDS, new LinkedBlockingQueue<>(2)) {
    ...省略其他
    //异步任务执行完成之后的钩子方法
    @Override
    protected void afterExecute(Runnable target, Throwable t)
    {
        ...省略其他
        //清空ThreadLocal实例的本地值
        startTime.remove();
    }
};
```

ThreadLocal升级版1: InheritableThreadLocal 可继承本地变量

什么是可继承本地变量InheritableThreadLocal (ITL) ?

我们先来看看 `InheritableThreadLocal`。

`InheritableThreadLocal` 这个类是 `ThreadLocal` 的一个变体，

`InheritableThreadLocal` 的特别之处在于，当一个线程派生出一个子线程时，子线程可以继承父线程中的值。

这在某些情况下特别有用，比如在进行请求处理或任务分派时。

官方注释如下：

This class extends `ThreadLocal` to provide inheritance of values from parent thread to child thread: when a child thread is created, the child receives initial values for all inheritable thread-local variables for which the parent has values. Normally the child's values will be identical to the parent's; however, the child's value can be made an arbitrary function of the parent's by overriding the `childValue` method in this class.

Inheritable thread-local variables are used in preference to ordinary thread-local variables when the per-thread-attribute being maintained in the variable (e.g., User ID, Transaction ID) must be automatically transmitted to any child threads that are created.

简单翻译如下：

- 此类扩展`ThreadLocal`以提供从父线程到子线程的值继承：
 - 创建子线程时，子线程将接收父线程具有值的所有可继承线程局部变量的初始值。通常情况下，子对象的值与父对象的值相同；
 - 但是，通过重写此类中的`childValue`方法，可以使子级的值成为父级的任意函数。
- 当变量中维护的每线程属性（例如，用户ID、事务ID）必须自动传输到创建的任何子线程时，可继承线程本地变量优先于普通线程本地变量。

`InheritableThreadLocal` 是 Java 中的一个类，它是 `ThreadLocal` 的一个变体，用于解决线程继承问题。

在 Java 中，每个线程都有自己的线程本地变量，称为 `ThreadLocal` 变量。然而，在使用线程池或创建线程组时，可能会遇到一个问题：子线程无法访问父线程的 `ThreadLocal` 变量，这就是线程继承问题。

`InheritableThreadLocal` 的作用就是解决这个问题。

- 当一个线程创建子线程时，子线程会继承父线程的 `InheritableThreadLocal` 变量的值。
- 这样，子线程就可以直接访问父线程的 `InheritableThreadLocal` 变量，而不需要手动传递值。

使用 `InheritableThreadLocal` 类似于使用普通的 `ThreadLocal`，你可以通过 `set()` 方法设置变量的值，通过 `get()` 方法获取变量的值。但是，`InheritableThreadLocal` 会在创建子线程时自动继承父线程的变量值。

来看一个非常简单的InheritableThreadLocal例子：

```
public class InheritableThreadLocalExample {  
    // 创建一个InheritableThreadLocal对象
```

```

private static InheritableThreadLocal<String> inheritableThreadLocal = new
InheritableThreadLocal<>();

public static void main(String[] args) {
    // 在父线程中设置值
    inheritableThreadLocal.set("父线程值");

    // 创建子线程
    Thread childThread = new Thread(() -> {
        // 子线程可以获取父线程设置的值
        System.out.println("子线程值: " + inheritableThreadLocal.get());
    });

    childThread.start();
}
}

```

在这个例子中，咱们在父线程中设置了一个值，然后在子线程中能够获取到这个值。

这就是 `InheritableThreadLocal` 的魔力所在。

```
threadLocal.remove();
```

这个做法可以确保 `ThreadLocal` 变量及时被清除，[避免内存泄漏](#)。

InheritableThreadLocal的基本使用

`InheritableThreadLocal` 扩展 `ThreadLocal` 以提供从父线程到子线程的值继承：创建子线程时，子线程将接收父线程具有值的所有可继承线程局部变量的初始值。

```

public class InheritableThreadLocalTest {

    private static final ThreadLocal<Integer> t1 = new InheritableThreadLocal<>
();

    public static void main(String[] args) throws Exception {
        t1.set(1);
        System.out.printf("当前线程名称: %s, main方法内获取线程内数据为: %s\n",
            Thread.currentThread().getName(), t1.get());
        fc();
        new Thread(InheritableThreadLocalTest::fc).start();
        Thread.sleep(1000L); // 保证下面fc执行一定在上面异步代码之后执行
        fc(); // 继续在主线程内执行，验证上面那一步是否对主线程上下文内容造成影响
    }

    private static void fc() {
        System.out.printf("当前线程名称: %s, fc方法内获取线程内数据为: %s\n",
            Thread.currentThread().getName(), t1.get());
    }

}

```

可以发现，子线程里已经可以获得父线程里的本地变量了。

当前线程名称：main，main方法内获取线程内数据为：1
当前线程名称：main，fc方法内获取线程内数据为：1
当前线程名称：Thread-0，fc方法内获取线程内数据为：1
当前线程名称：main，fc方法内获取线程内数据为：1

InheritableThreadLocal的原理分析

通过上述的测试代码，基本可以确定父线程的TLMaP被传递到了下一级，那么我们基本可以确认ITL是TL派生出来专门解决线程本地变量父传子问题的，那么下面通过源码来分析一下ITL到底是怎么完成这个操作的。

先来了解下Thread类，上节说到，其实最终线程本地变量是通过TLMaP存储在Thread对象内的，那麽来看下Thread对象内关于TLMaP的两个属性：

```
/* ThreadLocal values pertaining to this thread. This map is maintained
 * by the ThreadLocal class. */
ThreadLocal.ThreadLocalMap threadLocals = null;

/*
 * InheritableThreadLocal values pertaining to this thread. This map is
 * maintained by the InheritableThreadLocal class.
 */
ThreadLocal.ThreadLocalMap inheritableThreadLocals = null;
```

Thread类里其实有两个TLMaP属性，第一个就是普通TL对象为其赋值，第二个则由ITL对象为其赋值，来看下TL的set方法的实现，这次针对该方法介绍下TL子类的相关方法实现：

```
// TL的set方法，如果是子类的实现，那么获取（getMap）和初始化赋值（createMap）都是ITL对象里的方法
// 其余操作不变（因为hash计算、查找、扩容都是TLMaP里需要做的，这里子类ITL只起到一个为Thread对象里哪个TLMaP属性赋值的作用）
public void set(T value) {
    Thread t = Thread.currentThread();
    // 获取TLMaP
    ThreadLocal.ThreadLocalMap map = getMap(t);
    if (map != null)
        map.set(this, value);
    else
        // 子类实现初始化赋值
        createMap(t, value);
}

// ITL里getMap方法的实现
ThreadLocal.ThreadLocalMap getMap(Thread t) {
    return t.inheritableThreadLocals; //返回的其实是Thread对象的inheritableThreadLocals属性
}

// ITL里createMap方法的实现
void createMap(Thread t, T firstValue) {
    // 也是给Thread的inheritableThreadLocals属性赋值
    t.inheritableThreadLocals = new ThreadLocal.ThreadLocalMap(this, firstValue);
}
```

而 `inheritableThreadLocals` 里的信息通过 `Thread` 的 `init` 方法是可以被传递下去的：

```
// 初始化一个Thread对象时的代码段（Thread类的init方法）
Thread parent = currentThread();
if (parent.inheritableThreadLocals != null){ //可以看到，如果父线程存在
inheritableThreadLocals的时候，会赋值给子线程（当前正在被初始化的线程）
    // 利用父线程的TlMap对象，初始化一个TlMap，赋值给自己的
inheritableThreadLocals（这就意味着这个TlMap里的值会一直被传递下去）
    this.inheritableThreadLocals =
        ThreadLocal.createInheritedMap(parent.inheritableThreadLocals);
}

// 看下TL里对应的方法
static ThreadLocal.ThreadLocalMap
createInheritedMap(ThreadLocal.ThreadLocalMap parentMap) {
    return new ThreadLocal.ThreadLocalMap(parentMap); //这里就开始初始化TlMap对
象了
}

// 根据parentMap来进行初始化子线程的TlMap对象
private ThreadLocalMap(ThreadLocal.ThreadLocalMap parentMap) {
    ThreadLocal.ThreadLocalMap.Entry[] parentTable = parentMap.table; //拿到
父线程里的哈希表
    int len = parentTable.length;
    setThreshold(len); // 设置阈值（具体方法参考上一篇）
    table = new ThreadLocal.ThreadLocalMap.Entry[len];

    for (int j = 0; j < len; j++) {
        ThreadLocal.ThreadLocalMap.Entry e = parentTable[j]; //将父线程里的
Entry取出
        if (e != null) {
            @SuppressWarnings("unchecked")
            ThreadLocal<Object> key = (ThreadLocal<Object>) e.get(); //获取
key
            if (key != null) {
                Object value = key.childValue(e.value); //获取value
                ThreadLocal.ThreadLocalMap.Entry c = new
ThreadLocal.ThreadLocalMap.Entry(key, value); //根据k-v重新生成一个Entry
                int h = key.threadLocalHashCode & (len - 1); //计算哈希值
                while (table[h] != null)
                    h = nextIndex(h, len); //线性探查解决哈希冲突问题（具体方法参
考上一篇）
                table[h] = c; //找到合适的位置后进行赋值
                size++;
            }
        }
    }
}

// ITL里的childValue的实现
protected T childValue(T parentValue) {
    return parentValue; //直接将父线程里的值返回
}
```

InheritableThreaLocal所带来的问题

看过上述代码后，现在关于ITL的实现我们基本上有了清晰的认识了，根据其实现性质，可以总结出在使用ITL时可能存在的问题：

`InheritableThreadLocal` 主要是要用于主子线程之间传递参数的，这种方式有一个问题，那就是必须要是主线程中手动创建的子线程才可以，但对于使用线程池等池化复用线程的执行组件的情况，很多时候线程都是通过线程池进行创建和复用的，这时候 `InheritableThreadLocal` 就无计可施了。

1. 线程不安全

如果说线程本地变量是只读变量不会受到影响，但是如果是可写的，那么任意子线程针对本地变量的修改都会影响到主线程的本地变量（本质上是同一个对象），参考上面的第三个例子，子线程写入后会覆盖掉主线程的变量，也是通过这个结果，我们确认了子线程TlMap里变量指向的对象和父线程是同一个。

2. 线程池中可能失效

按照上述实现，在使用线程池的时候，ITL会完全失效，因为父线程的TlMap是通过init一个Thread的时候进行赋值给子线程的，而线程池在执行异步任务时可能不再需要创建新的线程了，因此也就不会再传递父线程的TlMap给子线程了。

ThreadLocal升级版2：TransmittableThreadLocal 可透传本地变量

什么是TransmittableThreadLocal (TTL) ?

`TransmittableThreadLocal` (TTL) 是阿里巴巴开源的一个Java库，用于解决 ThreadLocal 在多线程环境下的一些问题，尤其是在使用线程池等场景下可能出现的问题。

与普通的 ThreadLocal 不同，TTL 具有以下特点：

- 线程池透传性**：在使用线程池执行任务时，TTL 可以透传 ThreadLocal 的值，确保线程能够正确访问前线程设置的 TransmittableThreadLocal 变量值。
- 线程池隔离性**：TTL 在多线程环境下能够确保每个线程都有独立的 TransmittableThreadLocal 值，避免了线程池重用线程时可能出现的数据污染问题。
- 资源自动清理**：TTL 支持自动清理 TransmittableThreadLocal 值，避免了可能导致内存泄漏的问题。
- 兼容性**：TTL 兼容原生 ThreadLocal 的语法和用法，可以直接替换原生 ThreadLocal 使用，而无需修改现有代码。

TTL 使用场景

TTL 通常用于需要在线程池中执行任务，并且需要在任务之间传递 ThreadLocal 值的场景。

例如，在 Web 应用中，可能需要在异步任务中访问当前用户的会话信息，而使用 TTL 可以确保子线程能够正确访问父线程设置的会话信息。

`TransmittableThreadLocal` 在使用线程池等池化复用线程的执行组件情况下，提供 ThreadLocal 值的传递功能（把**任务提交给线程池时**的 ThreadLocal 值传递到**任务执行时**），解决异步执行时上下文传递的问题。

先来看一个例子：

```
public class TransmittableThreadLocalTest {  
  
    private static final ThreadLocal<Integer> t1 = new  
        TransmittableThreadLocal<>();  
}
```

```

public static void main(String[] args) {

    new Thread(() -> {

        String mainThreadName = "main_01";

        t1.set(1);

        new Thread(() -> {
            sleep();
            System.out.printf("本地变量改变之前(1), 父线程名称-%s, 子线程名称-%s,
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        new Thread(() -> {
            sleep();
            System.out.printf("本地变量改变之前(1), 父线程名称-%s, 子线程名称-%s,
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        new Thread(() -> {
            sleep();
            System.out.printf("本地变量改变之前(1), 父线程名称-%s, 子线程名称-%s,
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        sleep(); //确保上面的会在t1.set执行之前执行
        t1.set(2); // 等上面的线程池第一次启用完了, 父线程再给自己赋值

        new Thread(() -> {
            sleep();
            System.out.printf("本地变量改变之后(2), 父线程名称-%s, 子线程名称-%s,
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        new Thread(() -> {
            sleep();
            System.out.printf("本地变量改变之后(2), 父线程名称-%s, 子线程名称-%s,
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        new Thread(() -> {
            sleep();
            System.out.printf("本地变量改变之后(2), 父线程名称-%s, 子线程名称-%s,
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        System.out.printf("线程名称-%s, 变量值=%s%n",
Thread.currentThread().getName(), t1.get());

    }).start();

    new Thread(() -> {

        String mainThreadName = "main_02";

        t1.set(3);

```

```

        new Thread() -> {
            sleep();
            System.out.printf("本地变量改变之前(3)，父线程名称-%s，子线程名称-%s，
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        new Thread() -> {
            sleep();
            System.out.printf("本地变量改变之前(3)，父线程名称-%s，子线程名称-%s，
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        new Thread() -> {
            sleep();
            System.out.printf("本地变量改变之前(3)，父线程名称-%s，子线程名称-%s，
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        sleep(); //确保上面的会在t1.set执行之前执行
        t1.set(4); //等上面的线程池第一次启用完了，父线程再给自己赋值

        new Thread() -> {
            sleep();
            System.out.printf("本地变量改变之后(4)，父线程名称-%s，子线程名称-%s，
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        new Thread() -> {
            sleep();
            System.out.printf("本地变量改变之后(4)，父线程名称-%s，子线程名称-%s，
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        new Thread() -> {
            sleep();
            System.out.printf("本地变量改变之后(4)，父线程名称-%s，子线程名称-%s，
变量值=%s%n", mainThreadName, Thread.currentThread().getName(), t1.get());
        }).start();

        System.out.printf("线程名称-%s，变量值=%s%n",
Thread.currentThread().getName(), t1.get());

    }).start();

}

private static void sleep() {
    try {
        Thread.sleep(1L);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
}

```

运行结果:

本地变量改变之后(4), 父线程名称-main_02, 子线程名称-Thread-14, 变量值=4
本地变量改变之前(1), 父线程名称-main_01, 子线程名称-Thread-5, 变量值=1
线程名称-Thread-1, 变量值=2
本地变量改变之前(1), 父线程名称-main_01, 子线程名称-Thread-3, 变量值=1
本地变量改变之后(2), 父线程名称-main_01, 子线程名称-Thread-11, 变量值=2
本地变量改变之前(3), 父线程名称-main_02, 子线程名称-Thread-6, 变量值=3
本地变量改变之后(4), 父线程名称-main_02, 子线程名称-Thread-12, 变量值=4
本地变量改变之后(4), 父线程名称-main_02, 子线程名称-Thread-10, 变量值=4
本地变量改变之前(3), 父线程名称-main_02, 子线程名称-Thread-8, 变量值=3
本地变量改变之前(3), 父线程名称-main_02, 子线程名称-Thread-4, 变量值=3
本地变量改变之前(1), 父线程名称-main_01, 子线程名称-Thread-7, 变量值=1
线程名称-Thread-2, 变量值=4
本地变量改变之后(2), 父线程名称-main_01, 子线程名称-Thread-9, 变量值=2
本地变量改变之后(2), 父线程名称-main_01, 子线程名称-Thread-13, 变量值=2

到这里, 通过上述例子, TTL的基本使用我们已经有了初步的了解,

下面我们来解析一下其内部原理, 看看TTL是怎么完成对ITL的优化的。

TransmittableThreadLocal的原理分析

TTL 本质上是一个InheritableThreadLocal, 意味着TTL具备ITL的功能:

```
public class TransmittableThreadLocal<T>
    extends InheritableThreadLocal<T> implements TtlCopier<T> {}
```

具体值是保存在线程内部的inheritableThreadLocals, 不涉及线程池时可以当成InheritableThreadLocal使用。

TransmittableThreadLocal重要核心属性holder:

```
// 1. holder 本质上是一个 InheritableThreadLocal(a *ThreadLocal*).
// 2. 内部的值的类型是 WeakHashMap<TransmittableThreadLocal<Object>, ?>.
//    2.1 这个 WeakHashMap 被当成一个 *Set* 使用:
//        这个WeakHashMap的value永远是null, 不会被使用到.
//    2.2 WeakHashMap 支持 *null* 作为value.
private static final
InheritableThreadLocal<WeakHashMap<TransmittableThreadLocal<Object>, ?>> holder
=
    new
InheritableThreadLocal<WeakHashMap<TransmittableThreadLocal<Object>, ?>>() {
    @Override
    protected WeakHashMap<TransmittableThreadLocal<Object>, ?>
initialValue() {
        return new WeakHashMap<>();
    }

    @Override
    protected WeakHashMap<TransmittableThreadLocal<Object>, ?>
childValue(WeakHashMap<TransmittableThreadLocal<Object>, ?> parentValue) {
        return new WeakHashMap<>(parentValue);
    }
};
```

上述holder的作用:

存放的是所有和当前线程产生关联的 TransmittableThreadLocal，通过holder可以找到Java进程中所有的TransmittableThreadLocal（即使用WeakHashMap收集线程中所有的TransmittableThreadLocal）。

ThreadLocal.ThreadLocalMap和WeakHashMap的区别？

1. 相同点：ThreadLocalMap.Entry和WeakHashMap都继承弱引用，弱引用的reference都指向key：ThreadLocalMap.Entry；
2. 不同点：ThreadLocalMap是一个用移位解决hash冲突的简易Map，而WeakHashMap是一个用链表解决hash冲突的简易Map；

TTL重写TL的get和set方法处理保存的值：

```
//下面的方法均属于TTL类
@Override
public final T get() {
    T value = super.get();
    if (disableIgnoreNullValueSemantics || value != null) addThisToHolder();
    return value;
}

@Override
public final void set(T value) {
    if (!disableIgnoreNullValueSemantics && value == null) {
        // may set null to remove value
        remove();
    } else {
        super.set(value);
        addThisToHolder();
    }
}

@Override
public final void remove() {
    removeThisFromHolder();// 从holder持有的map对象中移除；
    super.remove();
}

private void removeThisFromHolder() {
    holder.get().remove(this);
}

private void addThisToHolder() {
    if (!holder.get().containsKey(this)) {
        // 从holder持有的map对象中添加
        holder.get().put((TransmittableThreadLocal<Object>) this, null); //
        weakHashMap supports null value.
    }
}
```

TTL里先了解上述的几个方法及对象，可以看出，单纯的使用TTL是达不到支持线程池本地变量的传递的，

通过第一部分的例子可以发现，除了要启用TTL，还需要通过**TtlExecutors.getTtlExecutorService**包装一下线程池才可以，

那么，下面就来看看在程序即将通过线程池异步的时候，TTL帮我们做了哪些操作（这一部分是TTL支持线程池传递的核心部分）：

首先打开包装类，看下execute方法在执行时做了些什么。

```
// 此方法属于线程池包装类ExecutorTtlWrapper
@Override
public void execute(@NonNull Runnable command) {
    executor.execute(TtlRunnable.get(command)); //这里会把Runnable包装一层，这是
    关键，有些逻辑处理，需要在run之前执行
}

// 对应上面的get方法，返回一个TtlRunnable对象，属于TtlRunnable包装类
@Nullable
public static TtlRunnable get(@Nullable Runnable runnable) {
    return get(runnable, false, false);
}

// 对应上面的get方法
@Nullable
public static TtlRunnable get(@Nullable Runnable runnable, boolean
    releaseTtlValueReferenceAfterRun, boolean idempotent) {
    if (null == runnable) return null;

    if (runnable instanceof TtlEnhanced) { // 若发现已经是目标类型了（说明已经被包
    装过了）直接返回
        // avoid redundant decoration, and ensure idempotency
        if (idempotent) return (TtlRunnable) runnable;
        else throw new IllegalStateException("Already TtlRunnable!");
    }
    return new TtlRunnable(runnable, releaseTtlValueReferenceAfterRun); //最
    终初始化
}

// 对应上面的TtlRunnable方法
private TtlRunnable(@NonNull Runnable runnable, boolean
    releaseTtlValueReferenceAfterRun) {
    this.capturedRef = new AtomicReference<Object>(capture()); //这里将捕获后
    的父线程本地变量存储在当前对象的capturedRef里
    this.runnable = runnable;
    this.releaseTtlValueReferenceAfterRun =
    releaseTtlValueReferenceAfterRun;
}

// 对应上面的capture方法，用于捕获当前线程（父线程）里的本地变量，此方法属于TTL的静态内部
    类Transmitter
    @NonNull
    public static Object capture() {
        Map<TransmittableThreadLocal<?>, Object> captured = new
        HashMap<TransmittableThreadLocal<?>, Object>();
        for (TransmittableThreadLocal<?> threadLocal : holder.get().keySet()) {
            // holder里目前存放的k-v里的key，就是需要传给子线程的TTL对象
            captured.put(threadLocal, threadLocal.copyValue());
        }
    }
}
```

```

        return captured; // 这里返回的这个对象，就是当前将要使用线程池异步出来的子线程，所
        继承的本地变量合集
    }

    // 对应上面的copyValue，简单的将TTL对象里的值返回（结合之前的源码可以知道get方法其实就是
    获取当前线程（父线程）里的值，调用super.get方法）
    private T copyValue() {
        return copy(get());
    }
    protected T copy(T parentValue) {
        return parentValue;
    }
}

```

结合上述代码，大致知道了在线程池异步之前需要做的事情，其实就是把当前父线程里的本地变量取出来，然后赋值给Rannable包装类里的capturedRef属性，

到此为止，下面会发生什么，我们大致上可以猜出来了，

接下来大概率会在run方法里，将这些捕获到的值赋给子线程的holder赋对应的TTL值，

那么我们继续往下看Rannable包装类里的run方法是怎么实现的：

```

//run方法属于Rannable的包装类TtlRunnable
@Override
public void run() {
    Object captured = capturedRef.get(); // 获取由之前捕获到的父线程变量集
    if (captured == null || releaseTtlValueReferenceAfterRun &&
        !capturedRef.compareAndSet(captured, null)) {
        throw new IllegalStateException("TTL value reference is released
        after run!");
    }

    /**
     * 重点方法replay，此方法用来给当前子线程赋本地变量，返回的backup是此子线程原来就有的
     本地变量值（原生本地变量，下面会详细讲），
     * backup用于恢复数据（如果任务执行完毕，意味着该子线程会归还线程池，那么需要将其原生
     本地变量属性恢复）
     */
    Object backup = replay(captured);
    try {
        runnable.run(); // 执行异步逻辑
    } finally {
        restore(backup); // 结合上面对于replay的解释，不难理解，这个方法就是用来恢复
        原有值的
    }
}
}

```

根据上述代码，我们看到了TTL在异步任务执行前，会先进行赋值操作（就是拿着异步发生时捕获到的父线程的本地变量，赋给自己），

当任务执行完，就恢复原生的自己本身的线程变量值。

下面来具体看这两方法：

```

//下面的方法均属于TTL的静态内部类Transmittable
@Nonnull

```

```

public static Object replay(@NonNull Object captured) {
    @SuppressWarnings("unchecked")
    Map<TransmittableThreadLocal<?>, Object> capturedMap =
        (Map<TransmittableThreadLocal<?>, Object>) captured; //使用此线程异步时捕获到的父线程
        里的本地变量值
    Map<TransmittableThreadLocal<?>, Object> backup = new
        HashMap<TransmittableThreadLocal<?>, Object>(); //当前线程原生的本地变量，用于使用完线
        程后恢复用

    //注意：这里循环的是当前子线程原生的本地变量集合，与本方法相反，restore方法里循环这个
    holder是指：该线程运行期间产生的变量+父线程继承来的变量
    for (Iterator<? extends Map.Entry<TransmittableThreadLocal<?>, ?>>
        iterator = holder.get().entrySet().iterator();
        iterator.hasNext(); ) {
        Map.Entry<TransmittableThreadLocal<?>, ?> next = iterator.next();
        TransmittableThreadLocal<?> threadLocal = next.getKey();

        backup.put(threadLocal, threadLocal.get()); // 所有原生的本地变量都暂时
        存储在backup里，用于之后恢复用

        /**
         * 检查，如果捕获到的线程变量里，不包含当前原生变量值，则从当前原生变量里清除掉，
         对应的线程本地变量也清掉
         * 这就是为什么会将原生变量保存在backup里的原因，为了恢复原生值使用
         * 那么，为什么这里要清除掉呢？因为从使用这个子线程做异步那里，捕获到的本地变量并
         不包含原生的变量，当前线程
         * 在做任务时的首要目标，是将父线程里的变量完全传递给任务，如果不清除这个子线程原
         生的本地变量，
         * 意味着很可能会影响到任务里取值的准确性。
         *
         * 打个比方，有ttl对象t1，这个t1在线程池的某个子线程里存在对应的值2，当某个主线
         程使用该子线程做异步任务时
         * t1这个对象在当前主线程里没有值，那么如果不进行下面这一步的操作，那么在使用该子
         线程做的任务里就可以通过
         * 该t1对象取到值2，不符合预期
         */
        if (!capturedMap.containsKey(threadLocal)) {
            iterator.remove();
            threadLocal.superRemove();
        }
    }

    // 这一步就是直接把父线程本地变量赋值给当前线程了（这一步起就刷新了holder里的值了，具
    体往下看该方法，在异步线程运行期间，还可能产生别的本地变量，比如在真正的run方法内的业务代码，再
    用一个t1对象设置一个值）
    setTtlValuesTo(capturedMap);

    // 这个方法属于扩展方法，ttl本身支持重写异步任务执行前后的操作，这里不再具体赘述
    doExecuteCallback(true);

    return backup;
}

// 结合之前Runnable包装类的run方法来看，这个方法就是使用上面replay记录下的原生线程变量做
    恢复用的
    public static void restore(@NonNull Object backup) {
        @SuppressWarnings("unchecked")

```

```

        Map<TransmittableThreadLocal<?>, Object> backupMap =
        (Map<TransmittableThreadLocal<?>, Object>) backup;
        // call afterExecute callback
        doExecuteCallback(false);

        // 注意，这里的holder取出来的，实际上是replay方法设置进去的关于父线程里的所有变量
        (结合上面来看，就是：该线程运行期间产生的变量+父线程继承来的变量)
        for (Iterator<? extends Map.Entry<TransmittableThreadLocal<?>, ?>>
        iterator = holder.get().entrySet().iterator();
            iterator.hasNext(); ) {
            Map.Entry<TransmittableThreadLocal<?>, ?> next = iterator.next();
            TransmittableThreadLocal<?> threadLocal = next.getKey();

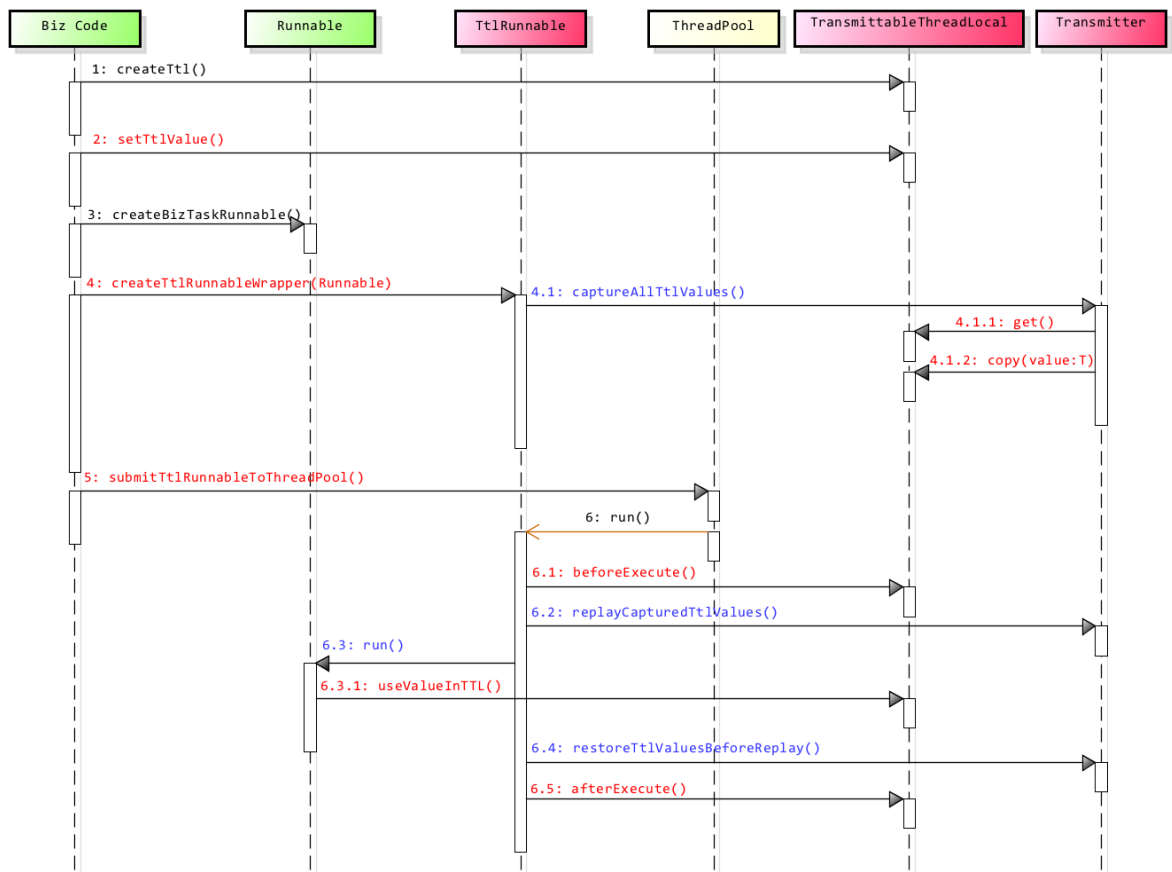
            /**
             * 同样的，如果子线程原生变量不包含某个父线程传来的对象，那么就删除，可以思考下，
             这里的清除跟上面replay里的有什么不同？
             * 这里会把不属于原生变量的对象给删除掉（这里被删除掉的可能是父线程继承下来的，也
             可能是异步任务在执行时产生的新值）
             */
            if (!backupMap.containsKey(threadLocal)) {
                iterator.remove();
                threadLocal.superRemove();
            }
        }

        // 同样调用这个方法，进行值的恢复
        setTtlValuesTo(backupMap);
    }

    // 真正给当前子线程赋值的方法，对应上面的setTtlValuesTo方法
    private static void setTtlValuesTo(@NonNull Map<TransmittableThreadLocal<?>,
    Object> ttlValues) {
        for (Map.Entry<TransmittableThreadLocal<?>, Object> entry :
        ttlValues.entrySet()) {
            @SuppressWarnings("unchecked")
            TransmittableThreadLocal<Object> threadLocal =
            (TransmittableThreadLocal<Object>) entry.getKey();
            threadLocal.set(entry.getValue()); //赋值，注意，从这里开始，子线程的
            holder里的值会被重新赋值刷新，可以参照上面ttl的set方法的实现
        }
    }
}

```

ok，到这里基本上把TTL比较核心的代码看完了，下面整理下整个流程，这是官方给出的时序图：



TransmittableThreadLocal的应用场景

TransmittableThreadLocal用来实现线程间的参数传递，经典应用场景如下：

1. 分布式跟踪系统 或 全链路压测（即链路打标）
2. 日志收集记录系统上下文
3. Session 级 Cache
4. 应用容器或上层框架跨应用代码给下层 SDK 传递信息

ThreadLocal、InheritableThreadLocal与TransmittableThreadLocal的比较

ThreadLocal、InheritableThreadLocal与TransmittableThreadLocal在Java中都是用于处理线程局部变量的工具，但它们在使用场景和特性上有所不同。

1. ThreadLocal

- **定义：** ThreadLocal是Java中一个非常重要的线程技术，它为每个线程提供了它自己的变量副本，使得线程间无法相互访问对方的变量，从而避免了线程间的竞争和数据泄露问题。
- **特性：**
 - **使用场景：** 适用于需要在线程内部存储和获取数据，且不希望与其他线程共享数据的场景。
 - **变量存储：** 提供了一种在线程内部存储变量的机制，每个线程可以独立地改变自己的副本，而不会影响到其他线程。
 - **线程隔离：** 通过为每个线程创建变量副本，ThreadLocal实现了线程间的数据隔离，提高了多线程程序的性能。
- **变量连续性：** 当线程切换时，ThreadLocal可以保持变量的连续性。

2. InheritableThreadLocal

- **定义：** InheritableThreadLocal是ThreadLocal的一个子类，它包含了ThreadLocal的所有功能，并扩展了ThreadLocal的功能。

- **特性：**
 - **线程继承：**允许父线程中的InheritableThreadLocal变量的值被子线程继承。当创建一个新的线程时，这个新线程可以访问其父线程中InheritableThreadLocal变量的值。
- **使用场景：**适用于需要在父线程和子线程之间传递数据的场景，如线程池中的任务传递等。

3. TransmittableThreadLocal

- **定义：**TransmittableThreadLocal是阿里巴巴开源的一个框架，用于解决在使用线程池等场景下，ThreadLocal变量无法跨线程传递的问题。
- **特性：**
 - **跨线程传递：**能够在多线程传递中保持变量的传递性，确保在父线程和子线程之间正确传递ThreadLocal变量。
 - **copy 方法**用于定制 任务提交给线程池时的 ThreadLocal 值传递到 任务执行时的拷贝行为，缺省传递的是引用。注意：如果跨线程传递了对象引用因为不再有线程封闭，与InheritableThreadLocal.childValue 一样，使用者/业务逻辑要注意传递对象的线程安全。
 - **protected 的 beforeExecute/afterExecute 方法**执行任务(Runnable/Callable)的前/后的生命周期回调。
- **使用场景：**适用于需要在线程池等场景下跨线程传递ThreadLocal变量的场景。

总结起来如下：

- **ThreadLocal**适用于线程内部的数据存储和访问，确保数据在线程间的隔离。
- **InheritableThreadLocal**适用于需要在父线程和子线程间传递数据的场景，实现数据的继承。
- **TransmittableThreadLocal**则是为了解决在使用线程池等场景下，ThreadLocal变量无法跨线程传递的问题，实现数据的跨线程传递。

在选择使用哪个类时，应根据具体的业务场景和需求进行权衡。同时，也需要注意在使用完这些类后，及时清理不再需要的数据，避免内存泄漏。

ThreadLocal和synchronized之间的比较

ThreadLocal和synchronized在Java中都是用于处理多线程问题的机制，但它们之间存在一些关键的区别。

1. 核心思想

- **ThreadLocal：**其核心思想是以空间换时间。它为每个线程提供了一个独立的变量副本，使得每个线程都可以访问和修改自己的变量副本，而不会影响到其他线程。由于每个线程操作的是自己的变量副本，因此多个线程可以同时访问该变量，且相互之间不会产生影响。这种机制主要用于保存线程私有数据、提高性能、管理线程特定的资源等场景。
- **synchronized：**其核心思想是以时间换空间。它确保同一时刻只有一个线程能够执行被synchronized修饰的代码块或方法，其他线程必须等待锁的释放。多个线程访问的是同一个变量，当多个线程同时访问该变量时，需要抢占锁，并且等待获取锁的线程释放锁，因此会消耗较多的时间。synchronized主要用于保护共享资源，防止竞态条件和数据不一致问题。

2. 应用场景

- **ThreadLocal**主要用于线程间的数据隔离，常见应用场景包括线程封闭（将对象封闭在单个线程中，避免线程安全问题）、保存线程上下文信息（如在Web开发中存储用户信息和请求参数）、数据库连接管理（确保每个线程获取到自己的数据库连接）以及线程池中的任务隔离等。
- **Synchronized**主要用于线程间的数据共享，常用于保护共享资源，如共享数据或对象，确保同一时间只有一个线程访问这些资源。此外，它还可以用于保护需要原子性执行的代码块，防止多线程并发执行导致的问题。

3. 性能和资源消耗

- **ThreadLocal**为每个线程创建变量副本，因此会消耗较多的内存。但由于线程间互不干扰，所以并发性能较高。
- **synchronized**则通过锁机制来控制线程对共享资源的访问，虽然节省了内存空间，但在多线程环境下可能会因为锁竞争而降低性能。

综上所述：

ThreadLocal和synchronized在解决多线程访问相同变量的冲突问题上各有其特点和适用场景。选择使用哪种机制应根据具体的业务需求、性能要求和资源限制来决定。

FastThreadLocal（FTL）的实现原理

ThreadLocal 是一个常用的工具类，它允许我们创建线程局部变量。这意味着每个线程都可以独立地改变自己的副本，而不会影响其他线程所持有的数据。

然而，ThreadLocal 在高并发环境下存在一些问题：

1. **内存占用**：每个 ThreadLocal 变量都会在每个线程中持有一个独立的副本，这可能导致大量的内存占用。
2. **性能开销**：创建和销毁这些线程局部变量会带来额外的性能开销。

Netty 是一个追求极致高性能的组件，Netty 的 FastThreadLocal 就是为了解决这些问题而诞生的。

什么是FastThreadLocal（FTL）？

Netty 的 FastThreadLocal 是一个高性能的线程本地变量实现，它与 Java 标准库中的 ThreadLocal 类似，但具有更高的性能和更低的内存消耗。FastThreadLocal 是由 Netty 框架提供的一个组件，旨在提供一种更快速、更高效的线程本地变量解决方案。

Netty 的 FastThreadLocal 使用了线程局部存储（Thread-Local Storage, TLS）的概念，但它通过一些优化手段减少了内存占用和性能开销。

1. **内存池化**：FastThreadLocal 使用了一个对象池来管理线程局部变量的实例，从而避免了频繁的创建和销毁操作。
Netty 对象池非常复杂，又非常重要，具体请参见尼恩的对象池视频。
2. **索引快速访问**：FastThreadLocal 使用了一个数组来存储每个线程的局部变量副本。通过计算线程的哈希码并作为索引，FastThreadLocal 可以快速地访问和修改线程局部变量的值。

FastThreadLocal 的原理与 ThreadLocal 类似，都是通过在每个线程中维护一个线程本地变量的副本来实现的。不同之处在于，FastThreadLocal 使用了一种更加高效的数据结构来管理线程本地变量，从而提高了访问速度和减少了内存消耗。

FastThreadLocal 如何使用

以下是一个简单的示例演示了如何使用 FastThreadLocal：

```
import io.netty.util.concurrent.FastThreadLocal;

public class FastThreadLocalExample {
    private static FastThreadLocal<String> threadLocal = new FastThreadLocal<>();

    public static void main(String[] args) throws InterruptedException {
        // 设置线程本地变量的值
    }
}
```

```

threadLocal.set("Hello, FastThreadLocal!");

// 创建并启动一个线程
Thread thread = new Thread(() -> {
    // 在子线程中获取线程本地变量的值
    String value = threadLocal.get();
    System.out.println("子线程获取到的线程本地变量的值为: " + value);
});
thread.start();
thread.join();

// 主线程获取线程本地变量的值
String value = threadLocal.get();
System.out.println("主线程获取到的线程本地变量的值为: " + value);

// 清除线程本地变量
threadLocal.remove();
}
}

```

在这个示例中，我们首先创建了一个 `FastThreadLocal` 实例，并在主线程中设置了一个线程本地变量的值。

然后，创建了一个子线程，在子线程中获取了该线程本地变量的值，并输出到控制台。

最后，主线程再次获取线程本地变量的值，并输出到控制台。

需要注意的是，在使用完毕后，我们需要调用 `remove()` 方法手动清除线程本地变量。

FastThreadLocal 的优势

1. **减少内存占用**：通过内存池化的方式，`FastThreadLocal` 避免了为每个线程局部变量创建和销毁实例的开销。
2. **提高性能**：通过索引快速访问线程局部变量的副本，`FastThreadLocal` 减少了查找和修改线程局部变量的时间。
3. **简化编程模型**：`FastThreadLocal` 提供了与标准 `ThreadLocal` 相似的 API，使得开发者可以在不改变编程习惯的前提下享受到性能提升。

FastThreadLocal 为什么快

`FastThreadLocal` 的实现与 `ThreadLocal` 非常类似，Netty 为 `FastThreadLocal` 量身打造了 `FastThreadLocalThread` 和 `InternalThreadLocalMap` 两个重要的类。

下面我们看下这两个类是如何实现的。

`FastThreadLocalThread` 是对 `Thread` 类的一层包装，每个线程对应一个 `InternalThreadLocalMap` 实例。

只有 `FastThreadLocal` 和 `FastThreadLocalThread` 组合使用时，才能发挥 `FastThreadLocal` 的性能优势。

首先看下 `FastThreadLocalThread` 的源码定义：

```
public class FastThreadLocalThread extends Thread {
    private InternalThreadLocalMap threadLocalMap;
    // 省略其他代码
}
```

可以看出 FastThreadLocalThread 主要扩展了 InternalThreadLocalMap 字段,

FastThreadLocalThread 主要使用 InternalThreadLocalMap 存储数据

注意, FastThreadLocalThread 不再是使用 Thread 中的 ThreadLocalMap。

所以想知道 FastThreadLocalThread 高性能的奥秘, 必须要了解 InternalThreadLocalMap 的设计原理。

上文中我们讲到了 ThreadLocal 的一个重要缺点, 就是 ThreadLocalMap 采用线性探测法解决 Hash 冲突性能较慢, 那么 InternalThreadLocalMap 又是如何优化的呢?

首先一起看下 InternalThreadLocalMap 的内部构造。

```
public final class InternalThreadLocalMap extends UnpaddedInternalThreadLocalMap
{
    private static final InternalLogger logger =
        InternalLoggerFactory.getInstance(InternalThreadLocalMap.class);
    private static final ThreadLocal<InternalThreadLocalMap> slowThreadLocalMap
        =
            new ThreadLocal<InternalThreadLocalMap>();
    private static final AtomicInteger nextIndex = new AtomicInteger();

    private static final int DEFAULT_ARRAY_LIST_INITIAL_CAPACITY = 8;
    private static final int ARRAY_LIST_CAPACITY_EXPAND_THRESHOLD = 1 << 30;
    // Reference:
    https://hg.openjdk.java.net/jdk8/jdk8/jdk/file/tip/src/share/classes/java/util/ArrayList.java#l229
    private static final int ARRAY_LIST_CAPACITY_MAX_SIZE = Integer.MAX_VALUE - 8;
    private static final int STRING_BUILDER_INITIAL_SIZE;
    private static final int STRING_BUILDER_MAX_SIZE;
    private static final int HANDLER_SHARABLE_CACHE_INITIAL_CAPACITY = 4;
    private static final int INDEXED_VARIABLE_TABLE_INITIAL_SIZE = 32;

    public static final Object UNSET = new Object();

    /** Used by {@link FastThreadLocal} */
    private Object[] indexedVariables;

    // Core thread-locals
    private int futureListenerStackDepth;
    private int localChannelReaderStackDepth;
    private Map<Class<?>, Boolean> handlerSharableCache;
    private IntegerHolder counterHashCode;
    private ThreadLocalRandom random;
    private Map<Class<?>, TypeParameterMatcher> typeParameterMatcherGetCache;
```

```

    private Map<Class<?>, Map<String, TypeParameterMatcher>>
typeParameterMatcherFindCache;

    // String-related thread-locals
    private StringBuilder stringBuilder;
    private Map<Charset, CharsetEncoder> charsetEncoderCache;
    private Map<Charset, CharsetDecoder> charsetDecoderCache;

    // ArrayList-related thread-locals
    private ArrayList<Object> arrayList;

    private BitSet cleanerFlags;

    /** @deprecated These padding fields will be removed in the future. */
    public long rp1, rp2, rp3, rp4, rp5, rp6, rp7, rp8;

    static {
        STRING_BUILDER_INITIAL_SIZE =

        SystemPropertyUtil.getInt("io.netty.threadLocalMap.stringBuilder.initialSize",
1024);
        logger.debug("-Dio.netty.threadLocalMap.stringBuilder.initialSize: {}",
STRING_BUILDER_INITIAL_SIZE);

        STRING_BUILDER_MAX_SIZE =
        SystemPropertyUtil.getInt("io.netty.threadLocalMap.stringBuilder.maxSize", 1024
* 4);
        logger.debug("-Dio.netty.threadLocalMap.stringBuil

der.maxSize: {}", STRING_BUILDER_MAX_SIZE);
    }

    // 省略其他代码
}

```

从 InternalThreadLocalMap 内部实现来看，与 ThreadLocalMap 一样都是采用 `数组` 的存储方式。

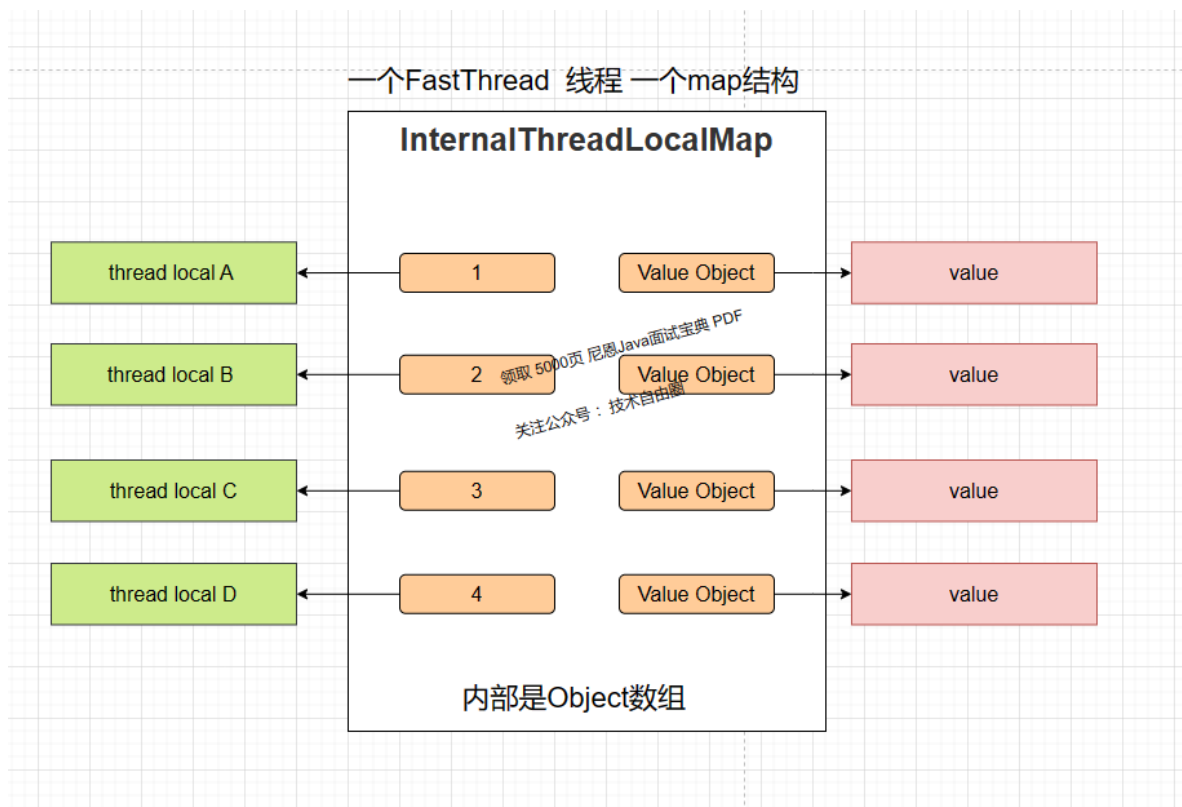
但是 InternalThreadLocalMap 并没有使用线性探测法来解决 Hash 冲突，而是另辟蹊径，使用数组替代 map。

简单来说，而是在 FastThreadLocal 初始化 的时候，为每一个本地变量，分配一个 `全局唯一的索引 index`，数组索引 index 的值采用原子类 `AtomicInteger` 保证顺序递增，

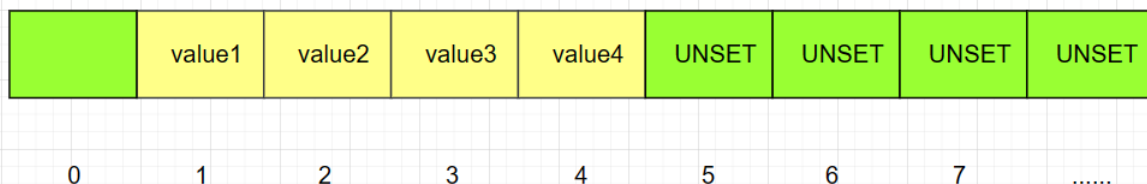
- 每一个本地变量，分配一个 `全局唯一的索引 index`。
- 这个数组索引 index 的值和本地变量绑定，通过调用 `InternalThreadLocalMap.nextVariableIndex()` 方法获得。

然后在读写数据的时候通过数组下标 index 直接定位到 FastThreadLocal 的位置，时间复杂度为 $O(1)$ 。

和普通的 ThreadLocalMap 相比，InternalThreadLocalMap 的大致内部结构，如下：

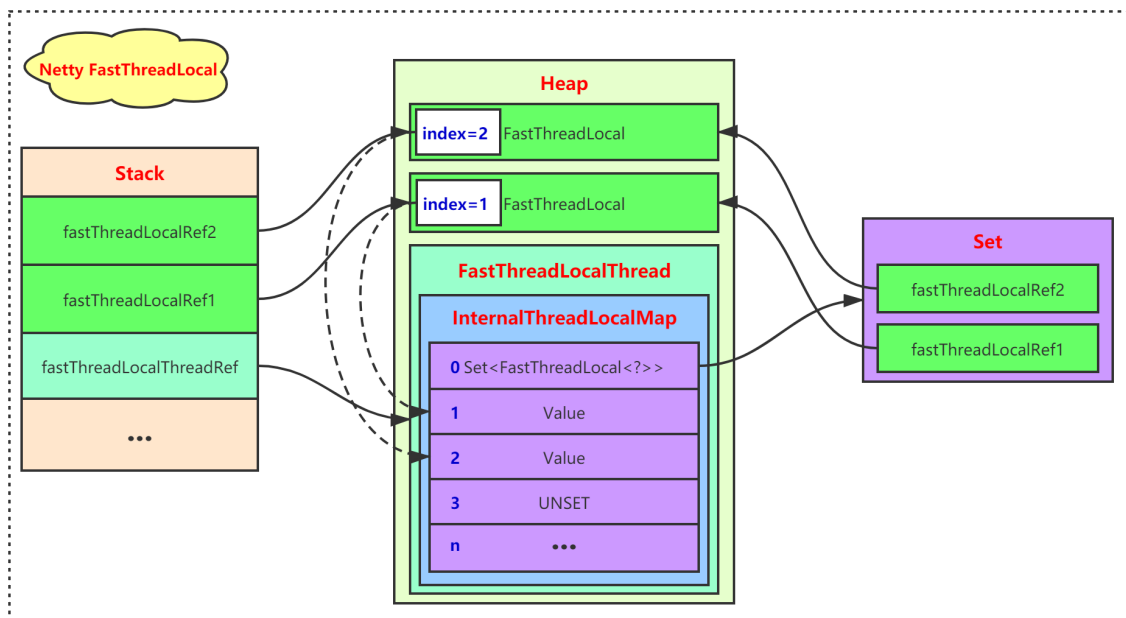


假设现在我们有一批数据需要添加到数组中，分别为 value1、value2、value3、value4，对应的 FastThreadLocal 在初始化的时候生成的数组索引分别为 1、2、3、4。如下图所示：



如果数组下标递增到非常大，那么数组也会比较大，所以 FastThreadLocal 是通过空间换时间的思想提升读写性能。

完整的 FastThreadLocal 结构分析，下面通过一幅具体的图，描述 InternalThreadLocalMap、index 和 FastThreadLocal 之间的关系。



1. InternalThreadLocalMap中并不是Entry的key-value结构, 而是Object数组
 2. 索引0位置存放FastThreadLocal的Set集合, 其他索引位置初始化为UNSET, 数据存入的时候更新为具体的Object
 3. FastThreadLocal中包含一个自增的index表示在InternalThreadLocalMap的数组中的索引位置
 4. Set<FastThreadLocal<?>>结构中存放FastThreadLocal的引用, 更容易解决内存泄漏的问题
- 通过上面 FastThreadLocal 的内部结构图, 我们对比下与 ThreadLocal 有哪些区别?

- FastThreadLocal 使用 Object 数组替代了 Entry 数组,
- Object[0] 存储的是一个Set集合,
- 从数组下标 1 开始都是直接存储的 value 数据, 不再采用ThreadLocal 的键值对形式进行存储。

FastThreadLocal 源码分析

get方法

源码中 **get()** 方法主要分为下面3个步骤处理:

1. 通过InternalThreadLocalMap.get()方法获取当前线程的InternalThreadLocalMap。
2. 根据当前线程的index 从InternalThreadLocalMap中获取其绑定的数据。
3. 如果不是缺省值UNSET, 直接返回; 如果是缺省值, 则执行initialize方法进行初始化。

```
public class FastThreadLocal<V> {
    // FastThreadLocal中的index是记录了该它维护的数据应该存储的位置
    private final int index;

    public final V get() {
        // 获取当前线程的InternalThreadLocalMap
        InternalThreadLocalMap threadLocalMap = InternalThreadLocalMap.get();
        // 根据当前线程的index从InternalThreadLocalMap中获取其绑定的数据
        Object v = threadLocalMap.indexedVariable(index);
        // 如果获取当前线程绑定的数据不为缺省值UNSET, 则直接返回; 否则进行初始化
        if (v != InternalThreadLocalMap.UNSET) {
            return (V) v;
        }
    }
}
```

```

        return initialize(threadLocalMap);
    }

    // 省略其他代码
}

```

下面我们继续分析一下InternalThreadLocalMap.get()方法的实现逻辑。

1. 首先判断当前线程是否是FastThreadLocalThread类型，如果是FastThreadLocalThread类型则直接使用fastGetMethod获取InternalThreadLocalMap，如果不是FastThreadLocalThread类型则使用slowGetMethod获取InternalThreadLocalMap兜底处理。
2. 兜底处理中的slowGetMethod方法会退化成JDK原生的ThreadLocal获取InternalThreadLocalMap。
3. 获取InternalThreadLocalMap时，如果为null，则会直接创建一个InternalThreadLocalMap返回。其创建过程中初始化一个32位长度的Object数组，并将其元素全部设置为缺省值UNSET。

```

public final class InternalThreadLocalMap extends UnpaddedInternalThreadLocalMap
{
    private static final int INDEXED_VARIABLE_TABLE_INITIAL_SIZE = 32;

    // 未赋值的Object变量（缺省值），当一个与线程绑定的值被删除之后，会被设置为UNSET
    public static final Object UNSET = new Object();

    // 存储绑定到当前线程的数据的数组
    private Object[] indexedVariables;

    // slowThreadLocalMap为JDK ThreadLocal存储InternalThreadLocalMap
    private static final ThreadLocal<InternalThreadLocalMap> slowThreadLocalMap
    =
        new ThreadLocal<InternalThreadLocalMap>();

    // 从绑定到当前线程的数据的数组中取出index位置的元素
    public Object indexedVariable(int index) {
        Object[] lookup = indexedVariables;
        return index < lookup.length? lookup[index] : UNSET;
    }

    public static InternalThreadLocalMap get() {
        Thread thread = Thread.currentThread();
        // 判断当前线程是否是FastThreadLocalThread类型
        if (thread instanceof FastThreadLocalThread) {
            return fastGet((FastThreadLocalThread) thread);
        } else {
            return slowGet();
        }
    }

    private static InternalThreadLocalMap fastGet(FastThreadLocalThread thread)
    {
        // 直接获取当前线程的InternalThreadLocalMap
        InternalThreadLocalMap threadLocalMap = thread.threadLocalMap();
        // 如果当前线程的InternalThreadLocalMap还未创建，则创建并赋值
        if (threadLocalMap == null) {
            thread.setThreadLocalMap(threadLocalMap = new
InternalThreadLocalMap());
        }
        return threadLocalMap;
    }
}

```

```

    }

    private static InternalThreadLocalMap slowGet() {
        // 使用JDK ThreadLocal获取InternalThreadLocalMap
        InternalThreadLocalMap ret = slowThreadLocalMap.get();
        if (ret == null) {
            ret = new InternalThreadLocalMap();
            slowThreadLocalMap.set(ret);
        }
        return ret;
    }

    private InternalThreadLocalMap() {
        indexedVariables = newIndexedVariableTable();
    }

    // 初始化一个32位长度的Object数组，并将其元素全部设置为缺省值UNSET
    private static Object[] newIndexedVariableTable() {
        Object[] array = new Object[INDEXED_VARIABLE_TABLE_INITIAL_SIZE];
        Arrays.fill(array, UNSET);
        return array;
    }

    // 省略其他代码
}

```

set方法

源码中 **set()** 方法主要分为下面3个步骤处理：

1. 判断value是否是缺省值UNSET，如果value不等于缺省值，则会通过 `InternalThreadLocalMap.get()` 方法获取当前线程的 `InternalThreadLocalMap`，具体实现3.2小节中 `get()` 方法已做讲解。
2. 通过 `FastThreadLocal` 中的 `setKnownNotUnset()` 方法将 `InternalThreadLocalMap` 中数据替换为新的value，并将当前的 `FastThreadLocal` 对象保存到待清理的Set中。
3. 如果等于缺省值UNSET或null（else的逻辑），会调用 `remove()` 方法，`remove()` 具体见后面的代码分析。

```

public class FastThreadLocal<V> {
    // FastThreadLocal初始化时variablesToRemoveIndex被赋值为0
    private static final int variablesToRemoveIndex =
        InternalThreadLocalMap.nextVariableIndex();

    public final void set(V value) {
        // 判断value值是否是未赋值的Object变量（缺省值）
        if (value != InternalThreadLocalMap.UNSET) {
            // 获取当前线程对应的InternalThreadLocalMap
            InternalThreadLocalMap threadLocalMap =
                InternalThreadLocalMap.get();
            // 将InternalThreadLocalMap中数据替换为新的value
            // 并将FastThreadLocal对象保存到待清理的Set中
            setKnownNotUnset(threadLocalMap, value);
        } else {
            remove();
        }
    }
}

```

```

private void setKnownNotUnset(InternalThreadLocalMap threadLocalMap, V
value) {
    // 将InternalThreadLocalMap中数据替换为新的value
    if (threadLocalMap.setIndexedVariable(index, value)) {
        // 并将当前的FastThreadLocal对象保存到待清理的Set中
        addToVariablesToRemove(threadLocalMap, this);
        variablesToRemove.add(variable);
    }
    // 省略其他代码
}

```

接下来我们看下InternalThreadLocalMap.setIndexedVariable方法的实现逻辑。

1. 判断index是否超出存储绑定到当前线程的数据的数组indexedVariables的长度，如果没有超出，则获取index位置的数据，并将该数组index位置数据设置新value。
2. 如果超出了，绑定到当前线程的数据的数组需要扩容，则扩容该数组并将它index位置的数据设置新value。
3. 扩容数组以index 为基准进行扩容，将数组扩容后的容量向上取整为 2 的次幂。然后将原数组内容拷贝到新的数组中，空余部分填充缺省值UNSET，最终把新数组赋值给 indexedVariables。

```

public final class InternalThreadLocalMap extends UnpaddedInternalThreadLocalMap
{
    // 未赋值的Object变量（缺省值），当一个与线程绑定的值被删除之后，会被设置为UNSET
    public static final Object UNSET = new Object();
    // 存储绑定到当前线程的数据的数组
    private Object[] indexedVariables;
    // 绑定到当前线程的数据的数组能再次采用x2扩容的最大量
    private static final int ARRAY_LIST_CAPACITY_EXPAND_THRESHOLD = 1 << 30;
    private static final int ARRAY_LIST_CAPACITY_MAX_SIZE = Integer.MAX_VALUE -
8;

    // 将InternalThreadLocalMap中数据替换为新的value
    public boolean setIndexedVariable(int index, Object value) {
        Object[] lookup = indexedVariables;
        if (index < lookup.length) {
            Object oldValue = lookup[index];
            // 直接将数组 index 位置设置为 value，时间复杂度为 O(1)
            lookup[index] = value;
            return oldValue == UNSET;
        } else { // 绑定到当前线程的数据的数组需要扩容，则扩容数组并数组设置新value
            expandIndexedVariableTableAndSet(index, value);
            return true;
        }
    }

    private void expandIndexedVariableTableAndSet(int index, Object value) {
        Object[] oldArray = indexedVariables;
        final int oldCapacity = oldArray.length;
        int newCapacity;
        // 判断可进行x2方式进行扩容
        if (index < ARRAY_LIST_CAPACITY_EXPAND_THRESHOLD) {
            newCapacity = index;
            // 位操作，提升扩容效率
            newCapacity |= newCapacity >>> 1;
            newCapacity |= newCapacity >>> 2;
            newCapacity |= newCapacity >>> 4;
            newCapacity |= newCapacity >>> 8;

```

```

        newCapacity |= newCapacity >>> 16;
        newCapacity++;
    } else { // 不支持x2方式扩容，则设置绑定到当前线程的数据的数组容量为最大值
        newCapacity = ARRAY_LIST_CAPACITY_MAX_SIZE;
    }
    // 按扩容后的大小创建新数组，并将老数组数据copy到新数组
    Object[] newArray = Arrays.copyOf(oldArray, newCapacity);
    // 新数组扩容后的部分赋UNSET缺省值
    Arrays.fill(newArray, oldCapacity, newArray.length, UNSET);
    // 新数组的index位置替换成新的value
    newArray[index] = value;
    // 绑定到当前线程的数据的数组用新数组替换
    indexedVariables = newArray;
}
// 省略其他代码
}

```

下面我们再继续看下FastThreadLocal.addToVariablesToRemove方法的实现逻辑。

1. 取下标index为0的数据（用于存储待清理的FastThreadLocal对象Set集合中），如果该数据是缺省值UNSET或null，则会创建FastThreadLocal对象Set集合，并将该Set集合填充到下标index为0的数组位置。
2. 如果该数据不是缺省值UNSET，说明Set集合已被填充，直接强转获取该Set集合。
3. 最后将FastThreadLocal对象保存到待清理的Set集合中。

```

private static void addToVariablesToRemove(InternalThreadLocalMap
threadLocalMap, FastThreadLocal<?> variable) {
    // 取下标index为0的数据，用于存储待清理的FastThreadLocal对象Set集合中
    Object v = threadLocalMap.indexedVariable(variablesToRemoveIndex);
    Set<FastThreadLocal<?>> variablesToRemove;
    if (v == InternalThreadLocalMap.UNSET || v == null) {
        // 下标index为0的数据为空，则创建FastThreadLocal对象Set集合
        variablesToRemove = Collections.newSetFromMap(new
IdentityHashMap<FastThreadLocal<?>, Boolean>());
        // 将InternalThreadLocalMap中下标为0的数据，设置成FastThreadLocal对象Set
集合
        threadLocalMap.setIndexedVariable(variablesToRemoveIndex,
variablesToRemove);
    } else {
        variablesToRemove = (Set<FastThreadLocal<?>>) v;
    }
    // 将FastThreadLocal对象保存到待清理的Set中
}

```

FastThreadLocal 的回收机制

在Netty中对于FastThreadLocal提供了三种回收机制：

自动： 使用FastThreadLocal执行一个被FastThreadLocalRunnable wrap的Runnable任务，在任务执行完毕后会进行ftl的清理。

手动： ftl和InternalThreadLocalMap都提供了remove方法，在合适的时候用户可以（有的时候也是必须，例如普通线程的线程池使用ftl）手动进行调用，进行显示删除。

自动：为当前线程的每一个ftl注册一个Cleaner，当线程对象不强可达的时候，该Cleaner线程会将当前线程的当前ftl进行回收。（netty推荐如果可以用其他两种方式，就不要再用这种方式，因为需要另起线程，耗费资源，而且多线程就会造成一些资源竞争，在netty-4.1.34版本中，已经注释掉了调用ObjectCleaner的代码。）

FastThreadLocal 在Netty中的应用

FastThreadLocal在netty中最重要的使用，就是**分配ByteBuf**。

基本做法是：每个线程都分配一块内存(PoolArena)，当需要分配ByteBuf时，线程先从自己持有的PoolArena分配，如果自己无法分配，再采用全局分配。

但是由于内存资源有限，所以还是会有多个线程持有同一块PoolArena的情况。

不过这种方式已经最大限度地减轻了多线程的资源竞争，提高程序效率。

具体的代码在**PoolByteBufAllocator的内部类PoolThreadLocalCache中：**

```
final class PoolThreadLocalCache extends FastThreadLocal<PoolThreadCache> {

    @Override
    protected synchronized PoolThreadCache initialValue() {
        final PoolArena<byte[]> heapArena = leastUsedArena(heapArenas);
        final PoolArena<ByteBuffer> directArena =
leastUsedArena(directArenas);

        Thread current = Thread.currentThread();
        if (useCacheForAllThreads || current instanceof
FastThreadLocalThread) {
            // PoolThreadCache即为各个线程持有的内存块的封装
            return new PoolThreadCache(
                heapArena, directArena, tinyCacheSize, smallCacheSize,
normalCacheSize,
                DEFAULT_MAX_CACHED_BUFFER_CAPACITY,
DEFAULT_CACHE_TRIM_INTERVAL);
        }
        // No caching so just use 0 as sizes.
        return new PoolThreadCache(heapArena, directArena, 0, 0, 0, 0, 0);
    }
}
```

和ThreadLocal相比， FastThreadLocal 的优势：

- **高效查找。**

FastThreadLocal 在定位数据的时候可以直接根据数组下标 index 获取，时间复杂度 O(1)。

而JDK原生的 ThreadLocal 在数据较多时哈希表很容易发生 Hash 冲突，线性探测法在解决 Hash 冲突时需要不停地向下寻找，效率较低。

此外，FastThreadLocal 相比 ThreadLocal 数据扩容更加简单高效，FastThreadLocal 以 index 为基准向上取整到 2 的次幂作为扩容后容量，然后把原数据拷贝到新数组。而 ThreadLocal 由于采用的哈希表，所以在扩容后需要再做一轮 rehash。

- **安全性更高。**

JDK 原生的 ThreadLocal 使用不当可能造成内存泄漏，只能等待线程销毁。

在使用线程池的场景下，ThreadLocal 只能通过主动检测的方式防止内存泄漏，从而造成了一定的开销。

然而 FastThreadLocal 不仅提供了 remove() 主动清除对象的方法，而且在线程池场景中 Netty 还封装了 FastThreadLocalRunnable，FastThreadLocalRunnable 最后会执行 FastThreadLocal.removeAll() 将 Set 集合中所有 FastThreadLocal 对象都清理掉。

说在最后：有问题找老架构取经

ThreadLocal 相关的面试题，是非常常见的面试题。

以上的内容，如果大家能对答如流，如数家珍，基本上面试官会被你震惊到、吸引到。最终，让面试官爱到“不能自己、口水直流”。offer，也就来了。

在面试之前，建议大家系统化的刷一波 5000 页《[尼恩Java面试宝典](#)》V174，在刷题过程中，如果有啥问题，大家可以来找 40 岁老架构师尼恩交流。

另外，如果没有面试机会，可以找尼恩来帮扶、领路。尼恩已经指导了大量的就业困难的小伙伴上岸。

前段时间，帮助一个 40 岁+就业困难小伙伴拿到了一个**年薪100W**的 offer，小伙伴实现了**逆天改命**。

部分历史案例

- [绝地逢生：9年经验自考](#)小伙伴，跟着尼恩狠卷3月硬核技术，面试机会爆表，2周后收3个 offer，满血复活
- [硬转架构：35岁被裁6个月 职业绝望，转架构急救上岸，DDD和3高项目太重要了](#)
- [逆天改命：3年经验2本，卷3个月涨薪60%，进准大厂\(得物、乐信、58集团等\)，年薪36W](#)
- [女35岁CRUD程序媛，12月被裁后，拿30个机会2个offer上岸，年底大裁员太不容易了](#)
- [38还有救吗？有。42岁被裁2年天快塌了，急救1个月上岸，成开发经理offer，起死回生](#)
- [被裁不慌，9年小伙1个月喜提年薪60W offer，做中间件架构，爽歪了](#)
- [12年小伙转架构，1小时蜕变收年薪55万offer，秘诀：GO+Java双栖架构](#)
- [年薪百万：40岁小伙被裁6个月，跟着尼恩猛卷3月硬核技术，100W年薪逆袭，上岸秘诀：首席架构/总架构](#)
- [不怕裁员，8年小伙去中年危机，秘诀：换架构师赛道，上得厅堂，下得厨房，越早越好](#)
- [被裁员后因祸得福，拿N+1后，逆袭30%，如何实现？](#)
- [逆天啦：4年crud小伙收shein+银行两优质offer，狠卷1月收年薪43万大涨30%](#)
- [原来，拿大厂offer有捷径.....](#)
- [太劲爆... 被裁4个月，38岁Android转Java，2个月提架构offer](#)
- [起死回生：8年小伙高中毕业+频繁跳槽，狠卷2月，提Java高级offer](#)
- [爽翻了：指导3轮，5年小伙收5大offer，涨50%，领路模式太牛](#)
- [降维攻击，37年大龄老伙喜提60W年薪offer，1个月顺利上岸](#)
- [架构速成：从一面杀到一次过，记7年女架构的速成经历](#)
- [极速拿offer：阿里P6被裁后极速上岸，1个月内喜提2优质offer\(含滴滴\)](#)
- [大逆袭：做化工12年，35岁转Java，被裁4个月，1个月收架构Offer，大龄跨行，超级牛](#)
- [被裁后炸爽：10年小伙12天火速上岸，反涨20%，爽爆了](#)
- [裁就裁，6年小伙60W极速上岸，白拿20W还游一圈拉萨，真香](#)
- [极速拿offer：被毕业3个月，11年经验小伙0.5个月极速拿offer，领路模式的巨大威力](#)
- [惊天大逆袭：8年小伙20天时间提75W年薪offer，逆袭50%，秘诀在这](#)

职业救助站

实现职业转型，极速上岸



关注**职业救助站**公众号，获取每天职业干货
助您实现**职业转型、职业升级、极速上岸**

技术自由圈

实现架构转型，再无中年危机



关注**技术自由圈**公众号，获取每天技术干货
一起成为牛逼的**未来超级架构师**

几十篇架构笔记、5000页面试宝典、20个技术圣经请加尼恩个人微信 免费拿走

暗号，请在 公众号后台 发送消息：领电子书

如有收获，请点击底部的"在看"和"赞"，谢谢

技术自由圈 技术圣经系列

技术自由圈 7个 学习圣经 PDF

- 1 《SpringCloud Alibaba 学习圣经》
v1 版，发布日期：2023年03月15日 377页 PDF
- 2 《Docker 学习圣经》
v2 版，发布日期：2023年03月12日 145页 PDF
- 3 《Kubernetes 学习圣经》
v1 版，发布日期：2023年04月11日 530页 PDF
- 4 《响应式圣经》
v2 版，发布日期：2023年02月14日 114页 PDF
- 5 《缓存之王 Caffeine 红宝书》
v2 版，发布日期：2022年11月30日 175页 PDF
- 6 《队列之王 Disruptor 红宝书》
v1 版，发布日期：2022年10月05日 75页 PDF
- 7 《Prometheus+grafna 红宝书》
v1 版，发布日期：2022年10月08日 100页 PDF

技术自由圈 3个 高并发圣经 PDF

- 8 《Java高并发核心编程 卷1 加强版：NIO、Netty、Redis、ZooKeeper》
加强版，发布日期：2023年01月01日 609页 PDF
- 9 《Java高并发核心编程 卷2 加强版：多线程、锁、JMM、JUC、高并发设计模式》
加强版，发布日期：2022年11月14日 448页 PDF
- 10 《Java高并发核心编程 卷3 加强版：亿级用户Web应用架构与实战》
加强版，发布日期：2022年11月14日 481页 PDF

技术自由圈 面试圣经 40个 面试题 PDF

- 11 《尼恩 Java 面试宝典》40个专题
v60 版，发布日期：2023年03月11日 4000页 PDF

领取方式：技术自由圈 公众号



硬核推荐：尼恩Java硬核架构班

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java
硬核架构班

已经发布

- ★ 《高性能RPC的基础实操之：从0到1开始IM撸一个IM》
- ★ 《分布式高性能RPC的基础实操之：千万级用户分布式IM实操- 含简历指导》
- ★ 《亿级用户超高并发秒杀实操- 含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，**2023夏招面试涨薪神器**
- ★ 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- ★ 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023夏招面试涨薪大神器
- ★ 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- ★ 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- ★ 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- ★ 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- ★ 《架构师实操篇：redis cluster 工业级高可用实操》
- ★ 《架构师实操篇：100W级别QPS日志平台实操》
- ★ 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》
- ★ 《超高并发场景100Wqps三级缓存组件原理和实操》
- ★ 《全链路异步超底层原理和实操：手写hystrix熔断+webflux+Lettuce+Dubbo》

规划中



左手大数据 (写入简历, 让简历 蓬荜生辉、金光闪闪)

HBASE + Flink + ElasticSearch 原理、架构、真刀实操



右手云原生 (写入简历, 让简历 蓬荜生辉、金光闪闪)

K8S + Devops + ServiceMesh 原理、架构、真刀实操

架构师实操篇: 基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架

架构师实操篇: go语言学习, 以及基于 go 手写 rpc 框架

架构师实操篇: 千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目

架构师实操篇: 工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目

特色

会员制

提供技术方向指导,
职业生涯指导, 少躺坑, 少弯路

简历指导

这个很重要,
对于挪窝涨薪来说

实操性

以上项目, 都是老架构师
在生产上实操过的项目

非水货

40岁老架构师, 不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

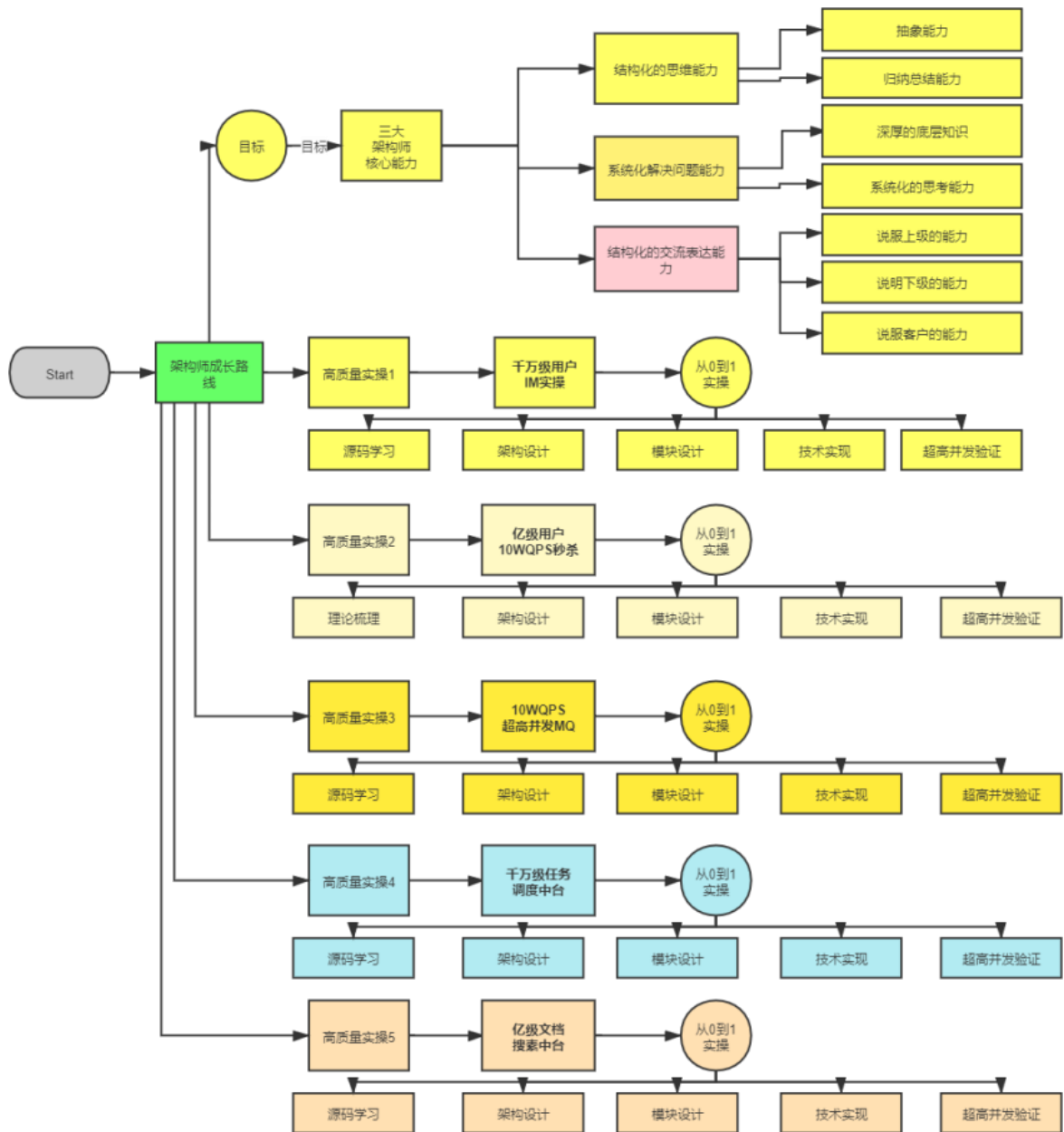
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



【样章】第 17 章：横扫全网 Rocketmq 视频第 2 部曲：工业级 rocketmq 高可用（HA）底层原理和实操

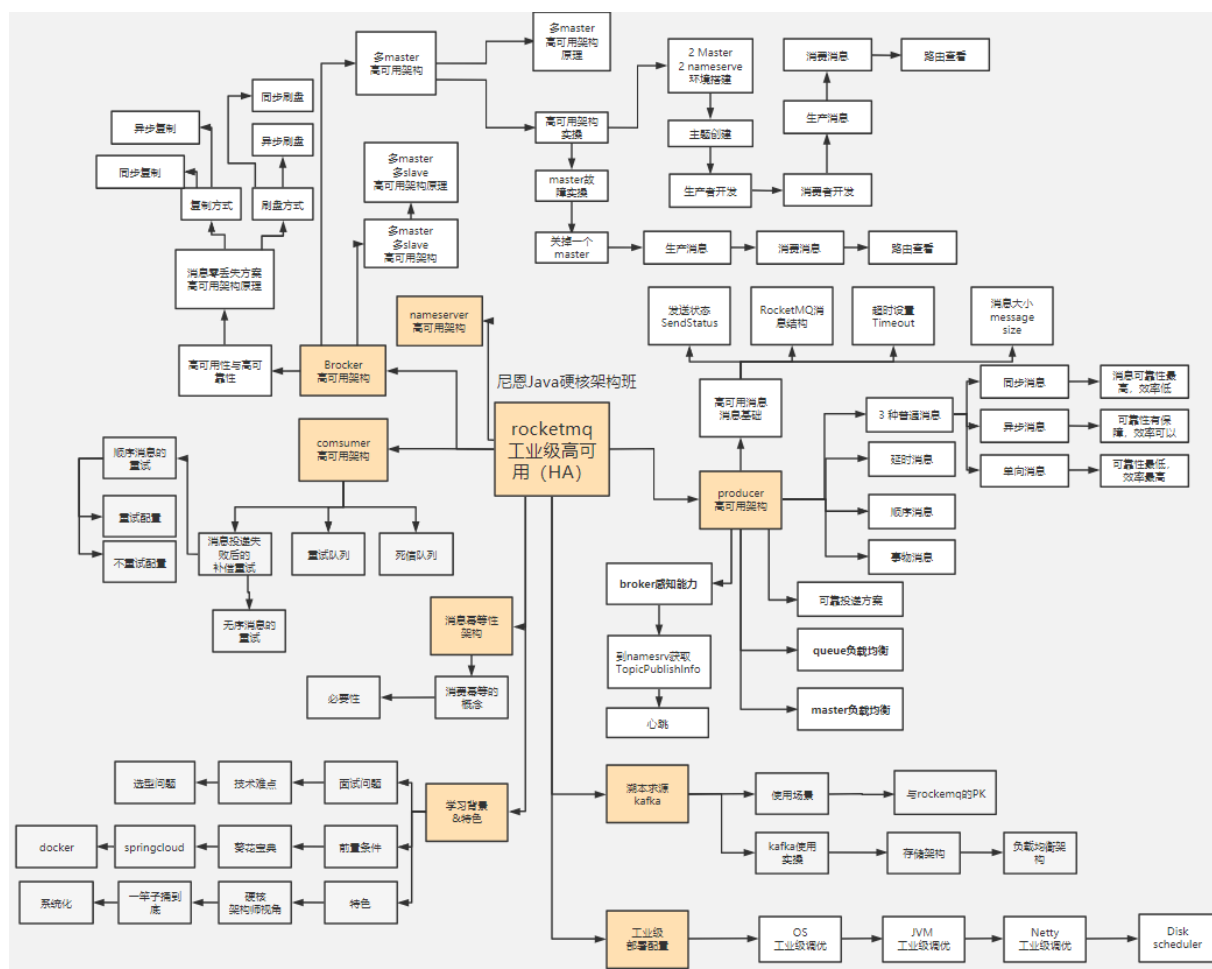
工业级 rocketmq 高可用底层原理，包含：消息消费、同步消息、异步消息、单向消息等不同消息的底层原理和源码实现；消息队列非常底层的主从复制、高可用、同步刷盘、异步刷盘等底层原理。

工业级 rocketmq 高可用底层原理和搭建实操，包含：高可用集群的搭建。

解决以下难题：

- 1、技术难题：RocketMQ 如何最大限度的保证消息不丢失的呢？RocketMQ 消息如何做到高可靠投递？
- 2、技术难题：基于消息的分布式事务，核心原理不理解
- 3、选型难题：kafka or rocketmq，该娶谁？

下图链接：<https://www.processon.com/view/6178e8ae0e3e7416bde9da19>



成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情：<http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新 最后发表 热门 精华

成功案例：[1057号卷王] 3年小伙拿到外企offer，薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例：[645号卷王] 4年经验卷王逆袭，被毕业后，反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例：[878号卷王] 小伙8年经验，年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例：通过尼恩的指导，小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例：[493号卷王] 5年小伙拿满意offer，就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例：[250号卷王] 就业极寒时代，收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例：[612号卷王] 就业极寒时代，从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例：[913号卷王] 热烈祝贺6年经验卷王，年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例：[959号卷王] 4年经验卷王，喜获百度、Boss直聘等N个优质offer，最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例：[529号卷王] 5年经验卷王喜收2大offer，最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例：[811号卷王] 热烈祝贺7年经验卷王，薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例：[287号卷王] 不惧大寒潮，卷王逆市收4 offer，涨30%，可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例：[1002号卷王] 5月份“被毕业”，改简历后，斩获顶级央企Offer，涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

1 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

1 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

1 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

1 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

1 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

1 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

1 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

1 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

1 卷王1号 超级版主 2022-3-25

❑ 成功案例: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

① 卷王1号 超级版主 2022-3-24

❑ 成功案例: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

① 卷王1号 超级版主 2022-3-21

❑ 成功案例: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

① 卷王1号 超级版主 2022-3-20

❑ 成功案例: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

① 卷王1号 超级版主 2022-3-16

❑ 简历案例: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 祝贺我圈两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

 尼恩 超级版主 2022-3-5

❑ 成功案例: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂Offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [449号卷王] 应届毕业卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

❑ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP含免费简历优化）

6年专科，2年翻4倍

2年从8K涨到35K

2021年从8K涨到22K

高并发 VIP76

老师，求助。

现在有两个满意的 offer，不知道怎么抉择。

一个是吉利，17k，大数据与 ai 部门。

另一个是一个平台，从零开始用 java 重写现在的项目，分布式架构，带团队，自己招人。22k，我觉得我说少了，我自己提的，然后今天发了 offer

呵呵，你太牛了

我也不好说

工资高的是个小公司，不到 50 人

感觉好事都被你占了

这一年半，真的谢谢您。

呵呵，相互交流，相互成长。

您写的书本，解决了我项目上很多问题。您在群里不厌其烦地告诉我们学习，也是我能坚持下来的重要因素，还有每次提问您都能解答疑惑，让我始终能戒骄戒躁。恩师，

**秘诀：
简历指导+ 狠狠卷**

2022年涨到35K

VIP76

解决了，限制 ip 频率。

谢谢老师

中午12:43

调整到了 35,加上这个月加班费，38

中午12:43

老师，我隔壁来了。

晚上8:23

大大的赞

老师你这路子是对的。我就跟着你学习思路和方法，还有教程走的。

我和你一样的兴奋和喜悦

记得咱们去年改简历的时候，还是 10k

这种提升，已经太令人震撼啦

是 8K...

20 年 4 月份转行，就一路跟着你学习





4年经验卷王逆袭 被毕业后，反涨24W

7月改简历

8月30日晒offer



小伙5月份"被毕业", 改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历

7月5日晒offer



卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历

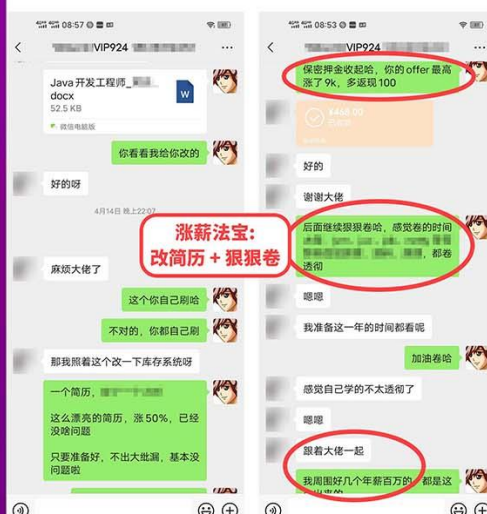
4月15日晒offer



卷王逆袭成功案例 6年小伙喜提4个Offer 最高涨9k, 年薪35W

4月14日改简历

5月17日晒offer



卷王逆袭成功案例

5年经验小伙收2个offer 最高涨薪8k，年薪42W

5月9日改简历

5月30日晒offer

秘诀:
简历指导+ 狠卷3高

以此为样
大家狠狠卷
打造最卷IT社群

卷王逆袭成功案例

非全日制 6年经验卷王 喜提3个Offer，年包30W

5月9日改简历

5月18日晒offer

面试法宝:
改简历+ 狠狠卷

卷王逆袭成功案例

寒五冻六之际卷王大逆袭 收3大offer，涨30%

5月17日改简历

5月27日晒offer

秘诀:
简历指导+ 狠卷3高

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历

5月12日晒offer

涨薪法宝:
改简历+ 狠狠卷

4年小伙喜收百度、Boss直聘等N个顶级Offer 最高涨幅100%

6月27日改简历 9月19日晒offer

**秘诀：
改简历 + 狠狠卷**

有个offer选择问题

boss直聘和小满之间，boss那边给的offer，工资比小满高，但是小满那边给的offer，福利比boss高，不知道该怎么选，如果你应该怎么选呀。

还有其他很多offer的，还有一个小满的，总体上看boss和小满之间选一个

了

boss比小满年包多7w以上，我这涨幅都接近百分之百了

太牛啦

按照你那个思路结合上次指导的内容改了几个点，发出去确实大上了很多，加上一些指标的说明确实会让人眼前一亮的感觉。

明天难时候有没了我和我看下哈，一是看两个点我实在不知道该怎么改了，二是我在推荐表达这方面确实不行，还得让你帮忙把把关。

卷王逆袭成功案例 4年卷王入收2个offer，涨50%

3月23日改简历 5月12日晒offer

尼恩老师，我的简历你看了吗，有什么建议？

ok

咱们开始吧

工作经历 这里不需要写描述

下边请描述，每段经历写什么

这个不用加粗

都是一样，加粗了反而没有效果

**涨薪法宝：
改简历 + 狠狠卷**

offer决策圈

n+1 电商erp，部门成熟，人数多，加班少。

n+2 做业务中台 内部系统，新部门，人数少，加班多

又搞到个offer

能搞定两个offer，不尴尬

现在很多小伙伴面试机会都没有

涨了多少钱

地点在哪里

涨50%的样子

忘记你工作几年啦，大概工作几年啦

小伙大三暑期很焦虑 跟着尼恩卷一年 校招斩获顶级央企Offer

去年5月19日加入VIP群 今年7月5日晒offer

**秘诀：
狠狠卷书+视频**

邀请你加入群聊

尼恩老师

我校招去华润电力控股有限公司了

跟着你卷了一年 大学顺便拿了几个国奖

不错不错，这是央企

放空中心

这太牛啦

其实拿到手的也就一个a类国一

尼恩老师 大三的暑期实习找不到 现在准备秋招应该没关系吧

看身边 总是很焦虑 自己算法这一块卡住

网盘里边有算法视频

去刷一刷吧

秋招来得及

谢谢大佬 不过经过几个月练习 看您写的书比之前轻松多了

趁着还是学生这段时间 慢慢把知识吃透

嗯哪，我的书，比较深

期待大佬的下一本书，已经迫不及待去学习了

小伙高中学历 薪酬涨120%

5月6日改简历 7月22日晒offer

你这块估计要送你668

模块的开发工作，就这个三个哈

哈哈，不用了老师，您真的讲了就行 光今天晚上辅导就值几千了

老弟很感谢您

还有很多其他的模块

面试官提问

就说是其他人做的

嗯哪，好

**秘诀：
改简历 + 狠狠卷**

之前你的工资是多少来的

翻了一翻

我得送你多少奖金来的

不知道，原价给老弟就行了

咱们得说这话呀

老弟拿着您的那个高开发改造亮点，所向披靡。

从从到尾给面试官讲明白

后面继续狠狠卷哈

卷王逆袭成功案例

非全日制卷王 面试3家
收2个offer 涨薪30%

4月13日改简历

4月21日晒offer

5年卷王喜收2大Offer

最高涨5K

5月19日改简历

9月13日晒offer

面试法宝:
改简历 + 面试题

卷王逆袭成功案例

3年经验卷王，涨60%

4月16日改简历

5月11日晒offer

卷王逆袭成功案例

双非二本小伙春招大翻身
喜提9大offer

2月22日改简历

4月13日晒offer

面试法宝:
改简历 + IM实操

涨薪法宝:
改简历 + 狠狠卷

9大offer 最高年薪30万

序号	公司	部门	岗位	薪资结构	总包
1	某公司	研发部	java后端开发	18.5k+14.5k+5k+200k/每月+500k/期权	>30w
2	某公司	研发部	java后端开发	16k+14k	22.4w
3	某公司	研发部	java后端开发	15k+15k+餐补+100k/每月	>22.5w
4	某公司	研发部	java后端开发	11k+13k	14.2w
5	某公司	研发部	java后端开发	14k	>16.8w
6	某公司	研发部	java后端开发	9k	9k
7	某公司	研发部	java后端开发	9k+餐补+社保+公积金	10k+餐补+房补
8	某公司	研发部	java后端开发	10k+餐补+房补	17w
9	某公司	研发部	java后端开发	10k+餐补+房补	17w

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>